Complejo Educativo Católico Dra. María Julia Hernández

Infraestructura Tecnológica y Servicios Informáticos

Docente: Yurian Idani García Batres

Modulo 3.1 Gestión De Base De Datos

Nombre del proyecto: Diseño e Implementación de un Sistema de Gestión de Inventario

Estudiantes:	Auto	COE
Gutiérrez Montoya, Ángel Gerardo #10	10	10
Guerra Ramos, Stephanie Jasmin #8	10	5
Hernández Chilulo, Carlos Alberto #10	10	3
Mancia Urrutia, Adrián Omar #15	10	0

Grado: 3°

Quezaltepeque lunes 23 de marzo del 2026

Índice

Introducción 4
Planteamiento del problema 2
Objetivos 3
objetivo general: 3
Objetivo específico: 3
Justificación 4
Marco Teórico 5
¿Qué es una base de datos? 5
¿Cuáles son los tipos de bases de datos? 5
¿Cuáles son las ventajas y desventajas de los tipos de bases de datos? 6
¿Qué es una tabla? 7
¿Cuáles son los tipos de tablas? 8
¿Cuáles son las ventajas y desventajas de estos tipos de tablas? 9
¿Qué es un campo? 13
¿Cuáles son los tipos de campos? 13
Campos Numéricos Enteros (Sin Decimales) 13
Campos Numéricos Decimales (Exactitud vs. Velocidad) 15
Campos de Texto y Cadenas (Alfanuméricos) 15
Campos de Fecha y Tiempo (Cronológicos) 16
Campos de Selección y Estructuras Modernas 16
¿Cuáles son las ventajas y desventajas de los tipos de campos? 17
Campos Numéricos Enteros (TINYINT, INT, BIGINT) 17
Campos Numéricos Decimales (DECIMAL, FLOAT, DOUBLE) 18
Campos de Texto de Longitud Fija (CHAR) 19
Campos de Texto de Longitud Variable (VARCHAR y TEXT) 19
Campos Temporales (DATE, DATETIME, TIMESTAMP) 20
Campos Especiales (ENUM y JSON) 20
¿Qué es un registro? 21
¿Cuáles son los tipos de registros? 21
Registros Maestros (Master Records) 22
Registros Transaccionales (Transaction Records) 22
Registros de Detalle o Relación (Detail/Join Records) 22
Registros de Auditoría o Log (Audit/Log Records) 23
Registros Históricos (Historical Records) 23

Registros Temporales (Temporary Records) 24
¿Cuáles son las ventajas y desventajas de los tipos de registros? 24
Registros Maestros (Master Records) 24
Registros Transaccionales (Transaction Records) 25
Registros de Detalle o Relación (Detail/Join Records) 25
Registros de Auditoría o Log (Audit/Log Records) 26
Registros Históricos (Historical Records) 26
Registros Temporales (Temporary Records) 26
¿Qué es una llave primaria? 27
¿Cuáles son los tipos de llaves primarias que existen? 27
La Llave Primaria Inteligente (Smart Key) 29
La Llave Primaria Candidata (Candidate Key) 30
La Llave Primaria Artificial (Surrogate Key) 30
¿Cuáles son las ventajas y desventajas de los tipos de llaves primarias? 31
¿Qué es una llave foránea? 33
¿Cuáles son los tipos de llaves foráneas? 33
¿Cuáles son las ventajas y desventajas de los tipos de llaves foráneas? 34
¿Qué es la normalización? 36
¿Cuáles son las formas normales que existen? 36
¿Cuáles son las ventajas y desventajas de la normalización? 38
Ventajas de la Normalización 38
Desventajas de la Normalización 39
¿Qué es un SGBD (Sistema de Gestión de Bases de Datos)? 39
¿Cuáles son los tipos de SGBD? 40
¿Cuáles son sus ventajas y desventajas de los SGBD? 42
Diseño de la Base de Datos 45
Diseño Conceptual 45
Diseño Lógico 45
Diseño Físico 46
Desarrollo del Proyecto 49
Pruebas del Sistema 55
Conclusiones 61
Recomendaciones 62
Bibliografía 63

Introducción

En la era de la información, una base de datos se define como un conjunto estructurado de datos almacenados digitalmente, los cuales están organizados de tal manera que un Sistema de Gestión de Bases de Datos (SGBD) pueda acceder, administrar y actualizar la información de forma eficiente. A diferencia de los registros manuales, una base de datos permite la interrelación lógica entre distintas entidades, garantizando que la información sea consistente, segura y escalable ante el crecimiento de cualquier organización.

En el presente proyecto, la implementación de una base de datos es de vital importancia, ya que surge como la solución técnica ante la situación problemática de Ferretería El Progreso S.A. de C.V., empresa que ha enfrentado pérdidas críticas de información debido a la falta de una estructura normalizada y de políticas de respaldo adecuadas. Al diseñar una infraestructura robusta, no solo se elimina la redundancia de datos, sino que se asegura la continuidad del negocio mediante protocolos de seguridad y planes de contingencia.

El sistema desarrollado consiste en una solución integral de gestión que abarca desde el modelado lógico (bajo las tres reglas de normalización) hasta la creación de una interfaz gráfica de usuario (GUI) funcional. Este software permite administrar de forma automatizada el inventario, el registro de clientes y el control de ventas, integrando restricciones de integridad como llaves foráneas y roles de usuario para proteger la sensibilidad de los datos. Con esta herramienta, la ferretería logra transitar hacia una gestión digital moderna, eficiente y, sobre todo, segura.

Planteamiento del problema

Actualmente, Ferretería El Progreso S.A. de C.V. enfrenta una crisis operativa debido a que su gestión de información depende de métodos tradicionales o estructuras digitales desorganizadas que carecen de normalización. Esta situación ha derivado en la pérdida constante de datos críticos, como niveles de inventario, saldos de clientes y registros históricos de ventas. La falta de políticas de respaldo y protocolos de seguridad pone en riesgo la estabilidad financiera de la empresa ante cualquier fallo técnico o error humano.

Es necesario crear esta infraestructura de base de datos porque el sistema actual impide el crecimiento de la ferretería; la redundancia de datos genera confusión en los precios y el desconocimiento del stock disponible afecta directamente el servicio al cliente. Por lo tanto, el problema que se pretende resolver es la ausencia de un sistema centralizado, seguro y escalable que garantice la integridad de la información y permita una toma de decisiones basada en datos reales.

Objetivos

objetivo general:

- Implementar un sistema funcional para la gestión y el control de los registros del sistema

Objetivo específico:

- Satisfacer correctamente las necesidades de SGBD exigidas para el correcto funcionamiento del sistema
- Implementar un uso adecuado de las llaves foráneas para la relación de los datos
- Realizar una interfaz gráfica intuitiva para facilitar el uso de esta

Justificación

La ejecución de este proyecto es fundamental para garantizar la supervivencia operativa de Ferretería El Progreso S.A. de C.V., ya que la transición de un sistema empírico a una infraestructura de base de datos robusta es el único camino para detener la pérdida de activos de información.

La importancia de este trabajo radica en la implementación de estándares de normalización y protocolos de seguridad que permitirán a la empresa manejar grandes volúmenes de registros con total fiabilidad y precisión técnica. Entre los principales beneficios se encuentran la optimización del inventario mediante una estructura que evita la redundancia, la garantía de confidencialidad a través de la configuración de roles de usuario (Admin, Editor, Lector) y la resiliencia de los datos mediante planes de respaldo automáticos que aseguran la continuidad del negocio ante cualquier fallo. Finalmente, el impacto de esta solución trasciende la eficiencia interna de la institución al modernizar sus procesos administrativos, traduciéndose en una mejora sustancial para la comunidad mediante un servicio al cliente más ágil, precios exactos y una atención basada en la disponibilidad real de los productos.

Marco Teórico

¿Qué es una base de datos?

Una base de datos es un sistema organizado y estructurado diseñado para recopilar, almacenar, gestionar y recuperar grandes volúmenes de información digital de manera eficiente. A diferencia de un simple archivo de texto o una lista desordenada, una base de datos permite que múltiples usuarios o aplicaciones accedan a los mismos datos de forma simultánea, garantizando que la información sea consistente, segura y fácil de buscar.

Para que una base de datos funcione, se apoya en un software especializado llamado Sistema de Gestión de Bases de Datos (DBMS por sus siglas en inglés, como MySQL, PostgreSQL o MongoDB), que actúa como intermediario entre el usuario y los datos físicos almacenados en el disco. En el modelo más utilizado, el relacional, la información se divide en tablas compuestas por filas (registros individuales) y columnas (categorías de datos), permitiendo establecer conexiones lógicas entre ellas para realizar consultas complejas mediante lenguajes como SQL. Siendo la infraestructura invisible que permite que desde una tienda en línea hasta una red social funcionen correctamente al recordar y procesar cada pieza de información que se les entrega.

¿Cuáles son los tipos de bases de datos?

Las bases de datos relacionales (SQL) representan el estándar de orden y estructura, utilizando tablas rígidas de filas y columnas que se conectan entre sí mediante llaves o identificadores únicos. Su prioridad absoluta es la integridad y consistencia, asegurando que los datos nunca se dupliquen ni se corrompan, lo que las hace indispensables en sistemas bancarios, de inventarios o registros académicos donde la precisión es vital.

Por el contrario, las bases de datos no relacionales (NoSQL) sacrifican la estructura rígida en favor de una flexibilidad y velocidad extremas, permitiendo almacenar información en formatos variados como documentos (JSON), grafos o pares de clave-valor. Son la columna vertebral de las redes sociales y el Big Data, ya que pueden escalar horizontalmente de forma masiva para procesar millones de datos desestructurados que cambian en tiempo real sin necesidad de esquemas previos.

En una categoría más específica, las bases de datos orientadas a objetos guardan la información directamente como "objetos", tal como se manejan en los lenguajes de programación modernos. Esto elimina la necesidad de traducir los datos a tablas, permitiendo que aplicaciones complejas en ingeniería o diseño manejen estructuras de datos intrincadas de forma mucho más natural y directa para el desarrollador.

Finalmente, las bases de datos distribuidas y en la nube (Cloud) se definen por su ubicación y disponibilidad, ya que reparten la información en múltiples servidores o centros de datos geográficos. Mientras que las distribuidas garantizan que el sistema siga funcionando, aunque falle un servidor físico, las bases de datos en la nube eliminan la necesidad de gestionar hardware, permitiendo que cualquier proyecto crezca en capacidad de almacenamiento al instante con solo un par de clics.

¿Cuáles son las ventajas y desventajas de los tipos de bases de datos?

Las bases de datos relacionales (SQL) tienen como mayor ventaja la integridad y consistencia de la información; gracias a sus reglas estrictas y al uso de transacciones ACID, garantizan que los datos nunca se corrompan ni se dupliquen. Son ideales para consultas complejas y reportes precisos. Sin embargo, su principal desventaja es la falta de flexibilidad, ya que cualquier cambio en la estructura (añadir una columna, por ejemplo) puede ser costoso y difícil de implementar en sistemas grandes. Además, su escalabilidad es principalmente vertical, lo que significa que para mejorar el rendimiento se requiere de un servidor más potente y costoso en lugar de muchos servidores pequeños.

Por el contrario, las bases de datos no relacionales (NoSQL) destacan por su flexibilidad y escalabilidad horizontal, permitiendo almacenar datos sin un esquema fijo y crecer fácilmente añadiendo más servidores económicos al clúster. Son extremadamente rápidas para leer y escribir volúmenes masivos de datos en tiempo real. Su desventaja crítica es que suelen sacrificar la consistencia inmediata por la velocidad; es decir, en un sistema muy grande, es posible que un dato recién actualizado tarde unos milisegundos en reflejarse en todos los nodos, lo que las hace menos recomendables para sistemas financieros donde cada centavo debe cuadrar al instante.

En el caso de las bases de datos en la nube (Cloud), la ventaja principal es la comodidad y disponibilidad, ya que el proveedor se encarga del mantenimiento, las copias de seguridad y de que el servicio nunca se caiga. Esto permite que los desarrolladores se enfoquen solo en el código. La desventaja es la dependencia de un tercero (si el proveedor sube los precios o tiene una falla masiva, el sistema se ve afectado) y que, a largo plazo y con tráficos muy altos, los costos mensuales pueden llegar a ser superiores a tener un servidor propio bien optimizado.

Finalmente, las bases de datos orientadas a objetos ofrecen la ventaja de una integración nativa con el software, eliminando la pérdida de rendimiento que ocurre al traducir objetos de programación a tablas de bases de datos. No obstante, su desventaja es la escasez de herramientas y talento especializado, ya que, al ser un mercado más pequeño, hay menos documentación y expertos disponibles en comparación con el ecosistema gigante de SQL o MongoDB.

¿Qué es una tabla?

Una tabla es la unidad fundamental donde se almacena la información dentro de una base de datos, se podría interpretar como una rejilla o cuadrícula organizada (muy similar a una hoja de Excel, pero con reglas mucho más estrictas). Su función es agrupar datos que pertenecen a una misma categoría, como "Productos", "Usuarios" o "Ventas", para que la computadora pueda encontrarlos, editarlos o borrarlos en milisegundos.

Para que una tabla funcione correctamente, se divide en tres elementos clave: las columnas (o campos), que definen qué tipo de información se va a guardar (por ejemplo: un nombre, un precio o una fecha); las filas (o registros), que son cada uno de los objetos individuales que llenan esa tabla (como los datos específicos de un martillo o un taladro); y la llave primaria, que es un identificador único (como un ID) que garantiza que no haya dos filas iguales. Gracias a esta estructura, las tablas pueden conectarse entre sí, permitiendo que una "Venta" sepa exactamente qué "Producto" se vendió y qué "Usuario" lo compró sin necesidad de repetir toda la información una y otra vez.

¿Cuáles son los tipos de tablas?

En el mundo de las bases de datos, las tablas no son todas iguales; se clasifican según la función que cumplen dentro del sistema. Los tipos de tablas más importantes son los siguientes:

- **Tablas Maestras (o de Entidad)**

Son las tablas principales que contienen los datos "base" del negocio. Almacenan información que no cambia constantemente y que representa objetos reales. Tomando como ejemplo el presente proyecto, estas son las tablas de productos, usuarios y roles. Sin estas tablas, el resto del sistema no tendría información con qué trabajar.

- **Tablas Transaccionales**

Estas tablas registran hechos o eventos que ocurren en un momento específico del tiempo. Suelen crecer muy rápido porque guardan cada operación que se realiza. La tabla de ventas es un gran ejemplo: cada vez que alguien compra un tornillo, se genera una nueva fila con la fecha, el monto y otros datos adicionales que se requieren.

- **Tablas de Relación (o Intermedias)**

Se utilizan para conectar dos tablas que tienen una relación de "muchos a muchos". Por ejemplo, si una Venta puede tener muchos Productos, y un Producto puede estar en muchas Ventas, se crea una tabla intermedia (como detalle_venta pero en este caso no se ha realizado) que solo guarda los IDs de ambos para unirlos. Sirven como un "puente" lógico.

- **Tablas de Configuración**

Son tablas pequeñas que guardan parámetros globales del sistema o listas de opciones predefinidas. La tabla configuración (donde se guardan los puntos y el valor de canje) es de este tipo. También podrían ser tablas de "Países", "Categorías de herramientas" o "Estados de envío".

- **Tablas Temporales**

Son tablas que el motor de la base de datos crea "al vuelo" para realizar cálculos complejos o guardar resultados intermedios. No están destinadas a vivir para siempre; una vez que la consulta o la sesión termina, la tabla se borra automáticamente para no ocupar espacio innecesario.

¿Cuáles son las ventajas y desventajas de estos tipos de tablas?

Cada tipo de tabla tiene un propósito específico dentro de un sistema de gestión de bases de datos. Elegir la correcta es lo que hace que una base de datos sea rápida y fácil de mantener.

- **Tablas Maestras**

Estas tablas son el "diccionario" de tu negocio. Su función es definir qué existe en tu sistema antes de que ocurra cualquier operación.

Ventajas:

La principal fortaleza de estas tablas es la eliminación de redundancia, basada en el principio de "escribir una sola vez"; esto significa que el nombre de un producto o el apellido de un cliente se registra en un único lugar, de modo que, si es necesario corregir una falta de ortografía o actualizar un nombre, el cambio se refleja instantáneamente en todas las facturas y reportes sin tener que editar miles de filas. A esto se suma la integridad de los datos, ya que al definir tipos de datos estrictos (como DECIMAL para precios o INT para el stock), el motor de la base de datos actúa como un guardia de seguridad que impide que alguien guarde texto por error donde debe ir un número, garantizando que los cálculos de inventario siempre sean exactos. Finalmente, ofrecen un esquema superior de seguridad y control, permitiéndote gestionar permisos granulares donde, por ejemplo, un empleado de ventas puede consultar la tabla para ver si hay existencia de un producto, pero tiene prohibido modificar el precio o el nombre, protegiendo así la información más crítica.

Desventajas:

El punto crítico de estas tablas es su rigidez estructural, ya que cualquier cambio en su diseño (como añadir un campo para "Marca" o "Categoría") requiere alterar la tabla completa, lo cual puede congelar el sistema brevemente si ya se posee demasiada información. Además, generan una dependencia restrictiva debido a las llaves foráneas; esto significa que no es posible borrar un producto de la lista si alguna vez se vendió, lo que te obliga a gestionar "borrados lógicos" (desactivar el producto) en lugar de eliminarlo físicamente para no romper el historial de ventas.

- **Tablas Transaccionales**

Son las que registran el "pulso" del negocio, guardando cada acción que genera un cambio en el estado del sistema.

Ventajas:

Su mayor beneficio es la trazabilidad e historial inalterable, ya que funcionan como una caja negra de avión que registra el momento exacto (fecha y hora), el monto y los actores de cada transacción, lo que es fundamental para realizar cierres de caja precisos y detectar posibles errores o fraudes. Además, son la fuente primaria para la inteligencia de negocios (BI), permitiendo transformar miles de filas de ventas en

información valiosa, como saber qué horas son las de mayor flujo en el sistema o qué temporada del año es la más rentable para comprar ciertos productos.

Desventajas:

La desventaja más evidente es el crecimiento masivo y degradación de velocidad, puesto que son tablas que acumulan miles de registros rápidamente, y si no se optimizan con índices, las búsquedas de facturas antiguas se vuelven extremadamente lentas. También presentan un riesgo de bloqueos de concurrencia, donde si muchos cajeros intentan registrar ventas pesadas simultáneamente, el motor de la base de datos podría hacer que algunos usuarios tengan que esperar milisegundos extra mientras se procesa la fila anterior.

- **Tablas de Relación o Intermedias**

Son las que permiten que la base de datos maneje la realidad de una compra: un cliente llevándose muchas cosas distintas en un solo ticket.

Ventajas:

Su ventaja principal es la escalabilidad y flexibilidad del modelo, permitiendo que una sola transacción sea "infinita" en contenido; sin esta tabla, se podría limitar el vender un solo artículo por factura, pero con ella es posible desglosar cada producto, su cantidad y su precio específico. Otra ventaja vital es la preservación del contexto histórico, ya que permiten guardar el precio de venta unitario de ese momento exacto; así, si el precio del cemento sube mañana en la tabla maestra, la tabla de relación "recuerda" a cuánto se lo vendiste al cliente ayer, manteniendo la contabilidad intacta.

Desventajas:

La principal contra es el aumento en la complejidad de las consultas, obligando al desarrollador a realizar múltiples uniones (JOIN) para poder reconstruir una factura, lo que exige más capacidad de procesamiento del servidor. Además, producen una explosión de registros, ya que por cada venta realizada se generan múltiples filas en esta tabla, lo que acelera el consumo de almacenamiento de manera mucho más agresiva que en las tablas maestras.

- **Tablas de Configuración**

Son las que guardan los parámetros que dictan cómo debe comportarse el software.

Ventajas:

Ofrecen una agilidad operativa total, permitiendo que el dueño del negocio ajuste variables críticas (como el porcentaje de IVA, el valor de los puntos de regalo o el límite de crédito) directamente desde la base de datos sin necesidad de llamar a un programador para que cambie el código fuente. Esto garantiza una centralización de reglas, asegurando que todos los cálculos del sistema utilicen el mismo valor actualizado en tiempo real.

Desventajas:

El riesgo más alto es que son un punto crítico de falla; al ser tablas pequeñas (a veces de una sola fila), un error humano al editar un valor o un borrado accidental puede dejar a todo el sistema calculando totales erróneos o impidiendo que se realicen ventas. Además, suelen carecer de historial de cambios, por lo que, si alguien cambia el valor de un parámetro, es difícil saber cuál era el valor anterior a menos que se implemente un registro de auditoría extra.

- **Tablas Temporales**

Son espacios de trabajo efímeros que el servidor crea en la memoria para tareas pesadas y puntuales.

Ventajas:

Su gran pro es el alto rendimiento y aislamiento, ya que al trabajar mayormente en la memoria RAM, permiten procesar grandes volúmenes de datos para reportes complejos sin "estorbar" ni poner lentas las tablas reales que los cajeros están usando en ese momento. Además, ofrecen limpieza automatizada, puesto que el sistema se encarga de borrarlas en cuanto la tarea termina, evitando que la base de datos se llene de archivos de "basura" o resultados intermedios.

Desventajas:

La mayor desventaja es su volatilidad absoluta, lo que significa que cualquier caída de conexión o reinicio del servicio borra los datos instantáneamente, haciéndolas inútiles para guardar información que deba persistir. También conllevan un riesgo de agotamiento de recursos, ya que, si se abusan de ellas con datos excesivamente grandes, pueden consumir toda la memoria RAM del servidor y provocar un colapso general del servicio de base de datos.

¿Qué es un campo?

En términos técnicos de ITSI, un Campo (también llamado Columna o Atributo) es la unidad mínima y atómica que define la estructura vertical de una tabla. No es solo un espacio vacío; es una regla de almacenamiento que obliga a cada registro a contener un dato específico bajo un formato estricto. Un campo tiene tres elementos que lo definen: un Nombre (ej. precio_venta), un Tipo de Dato (ej. DECIMAL) y una Restricción (ej. NOT NULL).

¿Cuáles son los tipos de campos?

En el diseño de bases de datos los tipos de campos (Data Types) son las categorías que definen qué clase de información puede almacenar cada columna. Ya que de esto depende la velocidad del servidor y la precisión de los cálculos.

Campos Numéricos Enteros (Sin Decimales)

Estos campos están diseñados para cálculos matemáticos puros y para indexación rápida. El motor de la base de datos reserva un espacio fijo de bits en la memoria RAM y en el disco.

TINYINT: Es el tipo de dato más pequeño, ocupando apenas 1 byte (8 bits). Su funcionamiento técnico se basa en el sistema binario: con 8 bits es posible representar 256 combinaciones. Si se define como SIGNED, el rango es de -128 a 127; si es UNSIGNED, se aprovecha todo el espacio para ir de 0 a 255. Es el campo ideal para "banderas" de estado o el Borrado Lógico en una base de datos, ya que procesar un solo byte es la operación más veloz que puede se puede realizar.

SMALLINT: Ocupa 2 bytes (16 bits). Su rango llega hasta 32,767 (o 65,535 en UNSIGNED). Se utiliza cuando el TINYINT se queda corto, como para el año de fabricación de un producto o el número de estante en un almacén. Es muy eficiente porque sigue siendo un tamaño de palabra que los procesadores modernos manejan en un solo ciclo de reloj.

MEDIUMINT: Un tipo intermedio que ocupa 3 bytes (24 bits). Permite llegar hasta los 8.3 millones. Es poco común, pero técnicamente brillante para ahorrar espacio en discos limitados cuando se sabe que los registros (como clientes de una ciudad específica) superan los 65 mil, pero nunca llegarán a los millones del INT estándar.

INT (o INTEGER): El estándar de oro. Ocupa 4 bytes (32 bits). Su rango alcanza los 2,147,483,647. Es el tipo de campo predilecto para las Llaves Primarias (IDs). Técnicamente, ofrece el mejor equilibrio entre capacidad de crecimiento y velocidad de procesamiento, ya que la mayoría de los sistemas operativos están optimizados para manejar datos de 32 bits de forma nativa.

BIGINT: El gigante de 8 bytes (64 bits). Su capacidad es astronómica, llegando a los 9 trillones. Solo se debe usar en casos de "Big Data", como identificadores de transacciones bancarias mundiales o registros de sensores que generan millones de datos por segundo, ya que usarlo innecesariamente duplica el consumo de memoria comparado con un INT.

Campos Numéricos Decimales (Exactitud vs. Velocidad)

Aquí se define cómo se comporta la base de datos ante los centavos y las fracciones.

DECIMAL(P, D) (o NUMERIC): Es un tipo de Punto Fijo. A diferencia de otros, el motor de la base de datos no lo guarda como un número binario simple, sino que utiliza una representación interna que garantiza precisión matemática absoluta. El parámetro P (Precisión) indica el número total de dígitos y D (Escala) el número de decimales. Si se define DECIMAL(10,2) para los precios de un sistema, el sistema siempre respetará los centavos sin errores de redondeo, lo cual es obligatorio para cualquier sistema contable o financiero.

FLOAT y DOUBLE: Son tipos de Punto Flotante. Ocupan 4 y 8 bytes respectivamente. Su ventaja es la velocidad extrema en cálculos científicos, pero su desventaja es que son "aproximados". A nivel de hardware, representar decimales en binario puede generar residuos ínfimos (ej. un 10.00 podría leerse internamente como 9.9999999). Por eso, son excelentes para gráficas o física, pero nunca para dinero.

Campos de Texto y Cadenas (Alfanuméricos)

Controlan cómo se almacena el lenguaje humano y cómo el servidor "salta" entre registros.

CHAR(n): Es de Longitud Estática. Esto significa que el disco duro reserva físicamente el número de bytes que estén definidos (n), sin importar si son usados o no. Si definimos CHAR(5) y guardamos solo la letra "A", el servidor gasta los 5 bytes de todos modos, rellenando el resto con espacios invisibles. Su ventaja técnica es el rendimiento de acceso aleatorio: como todos los registros miden lo mismo, el servidor sabe exactamente en qué byte empieza el siguiente dato, lo que hace que las búsquedas sean mucho más rápidas que en un VARCHAR. Es ideal para códigos fijos, siglas de países (SV, US) o hashes de contraseñas.

VARCHAR(n): Es de Longitud Dinámica. Si definimos VARCHAR(255) y guardamos "Pala", el sistema solo ocupará 4 bytes de texto más 1 o 2 bytes adicionales de "prefijo" que le indican al motor cuánto mide esa cadena específica. Es mucho más eficiente para ahorrar espacio en disco en campos como nombres o direcciones, aunque el

servidor debe hacer un pequeño esfuerzo extra para calcular dónde termina cada dato.

TEXT (TINYTEXT, MEDIUMTEXT, LONGTEXT): Estos campos son especiales porque no se guardan dentro de la tabla principal. El registro de la tabla solo contiene un "puntero" (una dirección de memoria). Esto permite que las tablas sigan siendo rápidas y ligeras al procesar miles de filas, dejando los textos pesados (como manuales o descripciones largas) en un área de almacenamiento secundaria.

Campos de Fecha y Tiempo (Cronológicos)

Permiten al servidor realizar aritmética de fechas (sumar días, restar meses).

DATE: Almacena exclusivamente el calendario (AAAA-MM-DD) usando 3 bytes. No registra horas, lo que lo hace perfecto para fechas de nacimiento o días festivos.

DATETIME: Registra tanto la fecha como la hora exacta (AAAA-MM-DD HH:MM:SS) usando 8 bytes. Es un valor absoluto: si grabas las 10:00 AM, siempre marcará esa hora sin importar la configuración del servidor.

TIMESTAMP: Ocupa solo 4 bytes y es el más inteligente. Almacena el tiempo como el número de segundos transcurridos desde 1970 (Era Unix). Su gran detalle técnico es que se convierte automáticamente según la zona horaria. Si se realiza una venta en El Salvador y el servidor está en Alemania, el dato se ajusta para que el reporte siempre sea coherente con la hora local del sistema.

Campos de Selección y Estructuras Modernas

Ayudan a evitar que el usuario ingrese datos que no tengan sentido.

ENUM: Es una lista de opciones predefinidas. Internamente, el servidor no guarda el texto, sino un número de índice (1, 2, 3...). Es extremadamente eficiente para campos como "Método de Pago" (Efectivo, Tarjeta, Bitcoin), ya que la base de datos rechaza automáticamente cualquier palabra que no esté en la lista, garantizando la Integridad Referencial.

JSON: Es un campo de nueva generación que permite guardar estructuras de datos flexibles. A diferencia de un simple TEXT, el campo JSON valida que el contenido tenga un formato correcto y permite realizar búsquedas internas sobre atributos específicos (por ejemplo, buscar solo el "Voltaje" dentro de un campo de especificaciones técnicas), uniendo la flexibilidad de las bases de datos NoSQL con la potencia de SQL.

¿Cuáles son las ventajas y desventajas de los tipos de campos?

En el mundo de las bases de datos, elegir el tipo correcto no es solo cuestión de "que funcione", sino de optimizar el rendimiento del servidor y garantizar que los datos del sistema sean exactos y seguros.

Campos Numéricos Enteros (TINYINT, INT, BIGINT)

Son los campos que almacenan números sin decimales. Se utilizan principalmente para contadores, IDs y cantidades fijas.

- **Ventajas:**

Su mayor fortaleza es la eficiencia computacional absoluta, ya que el procesador (CPU) puede realizar millones de comparaciones y sumas por segundo con estos datos, lo que hace que las búsquedas por ID sean casi instantáneas. Al ocupar un espacio fijo (desde 1 hasta 8 bytes), permiten una indexación veloz, optimizando el acceso al disco duro. Además, ofrecen una validación de entrada estricta, impidiendo que alguien ingrese accidentalmente texto o decimales en un campo de "Existencias", manteniendo la integridad del inventario.

- **Desventajas:**

El inconveniente principal es el riesgo de desbordamiento (Overflow); si se opta por un TINYINT (máximo 255) y el sistema crece a 300 productos, el sistema lanzará un error crítico y dejará de funcionar. También presentan el problema del desperdicio de memoria por sobredimensionamiento, donde usar un BIGINT para una edad o una cantidad pequeña reserva 7 bytes innecesarios por cada fila, lo que en tablas grandes ralentiza las copias de seguridad y satura la memoria RAM.

Campos Numéricos Decimales (DECIMAL, FLOAT, DOUBLE)

Son los encargados de manejar las fracciones, esenciales para los precios y medidas de un sistema.

- **Ventajas:**

La ventaja crítica del tipo DECIMAL es la precisión matemática exacta, ya que almacena los números como una cadena de dígitos, eliminando por completo los errores de redondeo en el cálculo del IVA o totales de facturas. Por otro lado, los tipos FLOAT y DOUBLE ofrecen una gran capacidad de rango y velocidad en cálculos científicos, permitiendo manejar números extremadamente grandes o pequeños con una carga mínima para el procesador.

- **Desventajas:**

La contraparte es el alto costo de procesamiento del tipo DECIMAL, el cual es más lento que los enteros porque el servidor debe procesar cada dígito individualmente para asegurar la exactitud. En el caso de FLOAT y DOUBLE, su gran desventaja es la imprecisión binaria, lo que significa que pueden perder micro-centavos en operaciones complejas, siendo un riesgo inaceptable para la contabilidad financiera.

Campos de Texto de Longitud Fija (CHAR)

Se usan para datos que siempre tienen el mismo tamaño, como siglas de países o códigos de productos estandarizados.

- **Ventajas:**

Su principal beneficio es el rendimiento superior en el acceso aleatorio. Como todos los registros miden exactamente lo mismo (por ejemplo, 5 bytes), el motor de la base de datos sabe matemáticamente dónde empieza y termina cada fila sin tener que "leer" el contenido previo. Esto reduce el esfuerzo del disco duro y acelera las búsquedas frecuentes en tablas maestras.

- **Desventajas:**

La desventaja obvia es el desperdicio masivo de espacio en disco (Padding). Si definimos un CHAR(100) y solo se escribe una palabra de 5 letras, el sistema gasta los 95 bytes restantes llenándolos con espacios vacíos. Esto hace que las tablas crezcan innecesariamente, haciendo que el servidor trabaje más de lo debido al mover datos pesados.

Campos de Texto de Longitud Variable (VARCHAR y TEXT)

Son los más comunes para nombres, direcciones y descripciones largas.

- **Ventajas:**

Su mayor virtud es la optimización inteligente del almacenamiento, ya que solo consumen el espacio del texto que realmente se escribe (más 1 o 2 bytes de control). Esto permite mantener bases de datos mucho más ligeras y eficientes en términos de espacio. En el caso de los campos TEXT, permiten una flexibilidad total para grandes volúmenes de datos, almacenándose fuera de la tabla principal para no ralentizar las consultas básicas.

- **Desventajas:**

El inconveniente técnico es la fragmentación y el costo de cálculo de longitud, ya que el servidor debe leer el prefijo de cada campo para saber cuánto mide antes de procesarlo, lo que lo hace ligeramente más lento que el tipo CHAR. Además, sufren el riesgo de truncado de datos, donde si un usuario intenta ingresar un texto más largo de lo definido, la base de datos cortará la información de forma irreversible.

Campos Temporales (DATE, DATETIME, TIMESTAMP)

Son los que permiten que un sistema tenga "memoria" del tiempo.

- **Ventajas:**

Permiten realizar aritmética de fechas nativa, lo que facilita crear reportes como "Ventas de los últimos 30 días" con una sola línea de código muy rápida. El tipo TIMESTAMP tiene la ventaja única de la gestión automática de zonas horarias, ajustando la hora de la venta según la ubicación del servidor, lo que garantiza auditorías coherentes a nivel global.

- **Desventajas:**

El problema principal es la rigidez de formato y validación, ya que cualquier error (como un mes "13" o un día "32") hará que la inserción de datos falle por completo. Además, el rango de TIMESTAMP es limitado (hasta el año 2038), lo que podría causar problemas de obsolescencia tecnológica en sistemas que necesiten proyectar fechas muy lejanas en el futuro.

Campos Especiales (ENUM y JSON)

Sirven para selecciones cerradas o estructuras de datos modernas.

- **Ventajas:**

El tipo ENUM garantiza una integridad de datos extrema, obligando a que el usuario solo elija opciones válidas (ej. 'Efectivo', 'Tarjeta'), lo que limpia los datos de errores ortográficos. El tipo JSON ofrece la flexibilidad de las bases de datos NoSQL,

permitiendo guardar características variables de productos (como el "voltaje" de un taladro) sin tener que crear cientos de columnas vacías.

- **Desventajas:**

La desventaja del ENUM es su dificultad de mantenimiento, ya que añadir una nueva opción requiere modificar la estructura de la tabla, lo que puede ser lento. Para el tipo JSON, el riesgo es la falta de validación de esquema, lo que puede permitir que se guarden datos desordenados o mal estructurados si no se controlan bien desde la aplicación.

¿Qué es un registro?

Un Registro es la unidad horizontal de una tabla que agrupa todos los campos necesarios para describir a un sujeto u objeto específico. Si los campos (columnas) son el "esqueleto" o el molde, el registro es la "carne" o el dato real que llena ese molde. En la teoría de conjuntos y el álgebra relacional, un registro se conoce como Tupla, y representa una relación única entre diferentes atributos.

Por ejemplo, en un nuestro sistema, un registro de la tabla Productos no es solo el nombre "Martillo", sino la unión indivisible de su ID (101), su Nombre (Martillo), su Precio ($12.50) y su Stock (50 unidades). Todos estos datos juntos forman un solo registro.

¿Cuáles son los tipos de registros?

Es fundamental entender que, aunque en una tabla todos los registros se ven iguales (filas horizontales), su función lógica y técnica varía según el papel que juegan en el sistema.

Registros Maestros (Master Records)

Son los datos fundamentales que definen la existencia de los objetos principales en un sistema. A nivel técnico, estos registros se caracterizan por su estabilidad y baja volatilidad, lo que significa que una vez insertados, sus cambios son mínimos (como corregir el nombre de una marca o actualizar una categoría). Se almacenan en tablas maestras y actúan como el origen de la verdad; por ejemplo, el registro de un "Taladro" existe una sola vez, pero es consultado miles de veces por otros procesos. Su importancia radica en que son la columna vertebral de la integridad referencial; si un registro maestro se corrompe o se elimina sin cuidado, todas las ventas, pedidos y movimientos de inventario asociados quedarían "huérfanos", perdiendo su significado lógico y arruinando las estadísticas de tu ferretería.

Registros Transaccionales (Transaction Records)

Son los registros que documentan un evento o acción específica que ocurre en un punto exacto del tiempo. Técnicamente, son los registros con mayor tasa de crecimiento (volumetría), ya que cada interacción (una venta, un abono, una devolución) genera una nueva fila. Estos registros no guardan datos descriptivos del producto, sino que utilizan Llaves Foráneas (Foreign Keys) para apuntar a los registros maestros, optimizando así el espacio en disco al no repetir nombres o marcas. Son vitales para la contabilidad y la auditoría, ya que permiten reconstruir el historial financiero; mientras que el registro maestro te dice qué vendes, los 10,000 registros transaccionales te dicen cuánto dinero ha ingresado realmente y en qué fechas.

Registros de Detalle o Relación (Detail/Join Records)

Estos registros son la solución técnica para las relaciones de "Muchos a Muchos" y son el corazón de una base de datos normalizada. Se ubican en tablas intermedias y su función es desglosar una transacción compleja; por ejemplo, en una sola venta (registro transaccional) un cliente puede llevar 5 productos distintos, lo que genera 5 registros de detalle. Cada uno de estos registros vincula el ID de la factura con el ID del producto y guarda datos que son exclusivos de ese momento, como el precio al

que se vendió en ese día o la cantidad de unidades. Su importancia técnica es que eliminan la redundancia, evitando la necesidad de repetir los datos generales de la factura (como el nombre del cliente o la fecha) por cada tornillo vendido.

Registros de Auditoría o Log (Audit/Log Records)

Son registros de control que funcionan como una "caja negra" del sistema, registrando quién, qué, cuándo y desde dónde se realizó cualquier modificación. A nivel técnico, suelen capturar el estado anterior y el estado nuevo de un dato (por ejemplo, el precio antes y después de un cambio), junto con la dirección IP y el ID del usuario responsable. Estos registros son oro puro para el administrador de sistemas, ya que son la única herramienta para resolver disputas, detectar fraudes internos o recuperar la lógica de un proceso tras un error humano. Sin ellos, sería imposible saber en qué segundo exacto falló una operación o quién alteró el inventario sin autorización.

Registros Históricos (Historical Records)

Son registros maestros o transaccionales que han cumplido su ciclo de vida activo pero que deben preservarse por razones legales o estadísticas. Técnicamente, aquí es donde se aplica el borrado lógico, marcando el registro con un estado "Inactivo" en lugar de eliminarlo físicamente del disco duro. Esto es crucial porque, si se elimina físicamente un producto que ya no está a la venta, todos los reportes de ventas de años anteriores fallarían al intentar buscar ese ID. Al mantenerlos como registros históricos, aseguramos que el historial financiero del sistema permanezca íntegro a través de las décadas, permitiendo comparar el crecimiento del negocio sin perder información valiosa del pasado.

Registros Temporales (Temporary Records)

Son registros de existencia efímera, diseñados para ser de "usar y tirar". A nivel de infraestructura, estos no suelen escribirse permanentemente en el almacenamiento principal, sino que se gestionan en la memoria RAM del servidor o en espacios de intercambio rápido (temp tables). Se crean automáticamente para facilitar procesos pesados, como el cálculo de un cierre de caja mensual o la generación de un reporte de inventario complejo que requiere unir varias tablas. Una vez que el proceso finaliza o la sesión del usuario se cierra, el servidor purga estos registros de forma automática. Su importancia técnica es crítica para el rendimiento, ya que permiten realizar cálculos masivos sin "ensuciar" ni ralentizar las tablas principales donde se operan las ventas diarias.

¿Cuáles son las ventajas y desventajas de los tipos de registros?

Es fundamental entender que cada tipo de registro tiene un "costo" y un "beneficio" dentro del servidor. No es recomendable llenar la base de datos de registros de auditoría sin esperar que el disco se llene, ni es posible tener solo registros maestros sin perder el rastro de las ventas.

Registros Maestros (Master Records)

Ventajas: Su mayor beneficio es la consistencia y normalización; al tener un solo registro para un producto (ej. "Taladro"), de tal modo que se evita tener que escribir el nombre y la marca en cada venta, ahorrando espacio masivo en disco. Además, facilitan las actualizaciones globales: si el proveedor cambia el nombre de una categoría, solo es necesario editar un registro maestro y el cambio se refleja en todo el sistema instantáneamente.

Desventajas: Su principal punto crítico es la dependencia absoluta de integridad; si un registro maestro se borra por error, todas las transacciones que dependían de él (ventas de hace meses o años) quedan corruptas o ilegibles. También requieren un mantenimiento riguroso, ya que cualquier error ortográfico en un maestro se replicará en miles de reportes y facturas.

Registros Transaccionales (Transaction Records)

Ventajas: Permiten una trazabilidad financiera y operativa exacta, proporcionando el poder de saber cuánto se ha vendido, a qué hora y quién fue el cajero. Son la base para generar reportes de inteligencia de negocios (BI) y proyecciones de ventas. Su estructura es ligera porque solo usan IDs (números) para referenciar a los maestros, lo que acelera su procesamiento.

Desventajas: El gran inconveniente es el crecimiento exponencial del almacenamiento (Volumetría); en un sistema con mucho movimiento, estos registros pueden llegar a los millones rápidamente, lo que hace que los backups sean pesados y que las consultas de búsqueda se vuelvan lentas si no se posee un buen plan de indexación y mantenimiento de hardware.

Registros de Detalle o Relación (Detail/Join Records)

Ventajas: Su fortaleza técnica es que resuelven la complejidad de las relaciones muchos a muchos de forma elegante. Permiten que una sola factura contenga múltiples artículos sin repetir los datos del cliente en cada fila. Además, permiten guardar datos históricos específicos, como el precio exacto al que se vendió un clavo ese día, aunque el precio maestro cambie al día siguiente.

Desventajas: Generan una mayor carga de procesamiento (CPU Overload) al realizar consultas, ya que el servidor debe hacer "uniones" (JOINs) entre tres o más tablas para mostrar una factura completa. Esto puede ralentizar el sistema si la lógica de programación no es eficiente o si el servidor tiene poca memoria RAM.

Registros de Auditoría o Log (Audit/Log Records)

Ventajas: Ofrecen una seguridad y transparencia total sobre lo que ocurre en la base de datos. Son la única forma de saber quién cometió un error o quién intentó hacer un fraude (como borrar una venta o cambiar un precio). Son indispensables para cumplir con normas de seguridad informática (como la ISO 27001).

Desventajas: Su desventaja es el alto consumo de recursos y escritura constante; cada vez que alguien hace un cambio pequeño, el servidor debe escribir un registro nuevo de auditoría, lo que genera un "estrés" adicional en el disco duro (I/O) y puede inflar el tamaño de la base de datos más rápido que las ventas mismas.

Registros Históricos (Historical Records)

Ventajas: Garantizan que la historia del negocio nunca se pierda. Gracias al borrado lógico, es posible desactivar productos viejos sin arruinar los reportes de años pasados. Mantienen la base de datos "limpia" para el día a día (filtros rápidos), pero disponible para análisis de largo plazo.

Desventajas: El problema es la acumulación de "datos muertos"; si no se gestionan bien, estos registros ocupan espacio precioso en el servidor que podría usarse para datos activos. Con el tiempo, obligan a realizar procesos de archivado externo (mover datos viejos a otra base de datos) para no afectar la velocidad del sistema principal.

Registros Temporales (Temporary Records)

Ventajas: Proporcionan una velocidad de procesamiento superior para tareas complejas, ya que al trabajar en la memoria RAM, el servidor no tiene que esperar a que el disco duro escriba o lea. Ayudan a que los reportes pesados no "bloqueen" las tablas reales que los cajeros están usando en ese momento.

Desventajas: Su mayor riesgo es la volatilidad total; si el servidor se reinicia o hay un corte de energía, estos registros desaparecen instantáneamente sin dejar rastro. Además, si no se programan para borrarse solos, pueden causar fugas de memoria (Memory Leaks), consumiendo toda la RAM del servidor hasta que este se congele.

¿Qué es una llave primaria?

Una Llave Primaria (abreviada como PK) es un campo, o una combinación de campos, que identifica de manera única e irrepetible a cada registro dentro de una tabla. Su existencia es lo que permite que una base de datos sea "relacional", ya que sirve como el punto de referencia exacto para conectar una tabla con otra.

Técnicamente, el motor de la base de datos (como MySQL) impone reglas estrictas sobre este campo: no puede estar vacío (NOT NULL) y no puede duplicarse (UNIQUE). Si se intenta ingresar un producto con un ID que ya existe, el servidor rechazará la operación para proteger la integridad de la información.

¿Cuáles son los tipos de llaves primarias que existen?

Entender los tipos de llaves Primarias es fundamental para decidir cómo se estructurará físicamente la base de datos de un sistema y entre ellas podemos hacer mención de las siguientes:

- **La Llave Primaria Natural (Natural Key)**

La Llave Primaria Natural es un identificador único que ya existe en el mundo real y posee un significado intrínseco fuera del sistema informático antes de ser registrado en una base de datos. A diferencia de las llaves artificiales, esta no es "inventada" por el software, sino que se extrae de los atributos propios del objeto o sujeto para garantizar su identidad.

En cuanto a su uso, se emplea cuando el objeto ya cuenta con un identificador universalmente aceptado y validado por una autoridad externa o por un estándar de industria. Los ejemplos más claros son el DUI para identificar a las personas, el VIN

(Número de Chasis) para el rastreo de vehículos, o el ISBN para la catalogación internacional de libros. Se utiliza directamente este valor como la identidad del registro en la tabla, permitiendo que la búsqueda en la base de datos coincida exactamente con el documento físico que el cliente tiene en su mano.

Para su implementación, este campo se define físicamente bajo tipos de datos de cadena de caracteres, como CHAR o VARCHAR, debido a que estos códigos suelen ser alfanuméricos (mezclan letras, números y guiones) y tienen una longitud específica definida por el estándar que los rige.

- **La Llave Primaria Compuesta (Composite Key)**

La Llave Primaria Compuesta es una estructura de identidad que se forma mediante la unión lógica de dos o más columnas de una misma tabla para crear un valor único combinado. En este esquema, ninguna de las columnas por sí sola puede garantizar la exclusividad del registro, pero la suma de todas ellas asegura que no exista una fila idéntica en la tabla.

En cuanto a su uso, se utiliza principalmente en tablas de relación o tablas intermedias (Muchos a Muchos) para modelar vínculos complejos entre entidades maestras. Por ejemplo, en una tabla de detalle_venta, se usa la combinación de id_venta y id_producto; el sistema permite que el ID de la venta se repita (porque una factura tiene varios productos) y que el ID del producto se repita (porque se vende a muchos clientes), pero prohíbe estrictamente que la combinación exacta de ambos aparezca más de una vez, evitando así duplicaciones erróneas en un mismo ticket.

Para su implementación, no se crea una columna nueva, sino que se define en la estructura del código SQL al final de la creación de la tabla mediante la instrucción específica: PRIMARY KEY (columna1, columna2, ...), vinculando los campos existentes bajo una única restricción de unicidad.

- **La Llave Primaria Global Única (UUID / GUID)**

La Llave Primaria Global Única (Universally Unique Identifier) es un identificador de 128 bits generado mediante algoritmos matemáticos complejos que garantizan que el código resultante sea único no solo en el servidor local, sino en cualquier dispositivo del planeta.

En cuanto a su uso, es el estándar en sistemas distribuidos, aplicaciones móviles o arquitecturas de microservicios donde los registros se crean en diferentes lugares (como teléfonos de distintos clientes o sucursales del sistema sin conexión a internet) y luego se sincronizan en una nube central. Su función es evitar colisiones de ID; gracias al UUID, dos sucursales pueden generar una venta al mismo milisegundo con la certeza absoluta de que nunca tendrán el mismo número de identificación al unirse en el servidor principal.

Para su implementación, se almacena generalmente como una cadena de texto de 36 caracteres (32 hexadecimales y 4 guiones) o, para optimizar el rendimiento, como un tipo de dato BINARY(16), lo que requiere que el motor de la base de datos o el lenguaje de programación (como PHP o Node.js) tenga una función específica para generar estos códigos aleatorios de alta seguridad.

La Llave Primaria Inteligente (Smart Key)

La Llave Primaria Inteligente es un identificador que contiene información descriptiva o categorizada del objeto codificada directamente dentro de su propio valor alfanumérico.

En cuanto a su uso, se emplea en la gestión de inventarios donde se requiere que el código sea "legible" para el ojo humano sin necesidad de consultar otros campos de la tabla. Por ejemplo, un código como HER-TAL-001 permite que un empleado del sistema sepa instantáneamente que se trata de un producto, de tipo Taladro y que es el correlativo 001. Fue muy popular en sistemas antiguos de almacén y logística para facilitar el etiquetado físico y la búsqueda manual en listas impresas.

Para su implementación, este campo se define obligatoriamente como un tipo de dato VARCHAR o TEXT, ya que requiere almacenar cadenas de texto con guiones o prefijos específicos que siguen una lógica de nomenclatura predefinida por la administración del negocio.

La Llave Primaria Candidata (Candidate Key)

La Llave Primaria Candidata es cualquier campo o combinación de campos dentro de una tabla que cumple con todos los requisitos técnicos para ser una Llave Primaria (es decir, que sus valores son únicos y nunca nulos), pero que el diseñador de la base de datos decidió no elegir como la llave principal. En una tabla profesional, puede haber varias "candidatas", pero solo una será la que se convertirá en la Primary Key.

En cuanto a su uso, estas llaves se identifican durante la fase de diseño de un sistema para garantizar que no haya duplicidad de datos importantes. Por ejemplo, en la tabla de Usuarios, poseemos dos candidatos: el ID_Usuario (Artificial) y el Correo Electrónico (Natural). Si seleccionamos el ID como Llave Primaria, el otro sigue siendo "Candidato". Su función es servir como respaldo de integridad; aunque no sean la llave principal, se les suele aplicar una restricción de UNIQUE para asegurar que nadie registre dos veces el mismo correo o el ID.

Para su implementación, se definen como campos normales de tipo INT, VARCHAR o CHAR, pero se les añade la restricción UNIQUE en el código SQL. Esto le dice al motor de la base de datos: "Esta no es la llave principal para las relaciones, pero prohíbe terminantemente que este valor se repita en otra fila".

La Llave Primaria Artificial (Surrogate Key)

La Llave Primaria Artificial es un identificador único que no tiene ninguna relación con los datos del mundo real del objeto, sino que es creado exclusivamente para el funcionamiento interno del sistema informático. A diferencia de una llave natural (como el DUI), este valor no aporta información sobre el producto o la persona; su única misión es existir para que el software pueda distinguir una fila de otra sin errores.

En cuanto a su uso, se utiliza como un contador interno de alta velocidad. El motor de la base de datos (como MySQL) asigna un número secuencial (1, 2, 3...) a cada registro nuevo mediante la propiedad AUTO_INCREMENT. Es el estándar profesional para identificar productos, clientes o ventas en sistemas donde la identidad del registro debe ser totalmente independiente de cualquier cambio externo. Por ejemplo, si un taladro cambia su nombre o su marca, su ID 50 siempre será el 50, lo que garantiza

que todas las facturas viejas sigan conectadas correctamente a ese producto sin romperse.

Para su implementación, se define generalmente como un campo de tipo INT (entero) o BIGINT (para sistemas con miles de millones de datos). Físicamente, es la llave que menos espacio ocupa en el disco duro y la que el procesador de un servidor puede comparar más rápido, lo que hace que la página web de un sistema así cargue mucho más rápido.

¿Cuáles son las ventajas y desventajas de los tipos de llaves primarias?

En la arquitectura de datos, cada llave ofrece un equilibrio distinto entre rendimiento y claridad lógica:

La Llave Primaria Artificial (Subrogada) tiene como principal ventaja su extrema eficiencia y estabilidad; al ser un número entero simple (INT), el procesador del servidor puede compararlos y ordenarlos a una velocidad increíble, y como no tiene significado en el mundo real, nunca cambia, protegiendo todas las relaciones. Su desventaja es que es totalmente anónima, lo que significa que un "ID 502" no te dice nada sobre el producto al mirar la tabla directamente, y requiere que el motor de la base de datos gestione un contador adicional (AUTO_INCREMENT).

Por otro lado, la Llave Primaria Natural ofrece la ventaja de la legibilidad y ahorro de espacio, ya que usa un dato que ya existe (como un DUI o código de barras) y no necesita crear columnas extras "inventadas". Sin embargo, su gran desventaja es su peligrosa inestabilidad; si la entidad que emite el código (el gobierno o un fabricante) decide cambiar el formato o reutilizar un número, se deberá de actualizar miles de registros relacionados, lo que puede provocar inconsistencias graves y caídas del sistema.

La Llave Primaria Compuesta tiene la ventaja de garantizar la lógica de negocio a nivel de base de datos, evitando por ejemplo que un mismo producto se registre dos veces en la misma factura. Su desventaja es la complejidad técnica, ya que hace que las consultas JOIN sean mucho más difíciles de escribir y que las llaves foráneas en otras tablas sean más "pesadas", consumiendo más memoria RAM al procesar relaciones entre tablas.

La Llave Primaria Candidata brinda la ventaja de añadir capas de seguridad de datos, permitiendo que campos como el correo electrónico sean únicos sin ser la llave principal. Su desventaja es que consume recursos adicionales de indexación; cada llave candidata que se protege con una restricción UNIQUE obliga al servidor a trabajar un poco más en cada inserción para verificar que no haya duplicados, lo que puede ralentizar ligeramente la entrada de datos masiva.

Para sistemas modernos, la Llave Primaria Global Única (UUID) tiene la ventaja de permitir la descentralización total; siendo posible crear registros en diez sucursales al mismo tiempo sin internet y estar seguro de que los IDs nunca chocarán al sincronizarse. Su desventaja es el pésimo rendimiento en discos duros tradicionales; al ser cadenas de 36 caracteres aleatorios, los índices crecen de forma desordenada (fragmentación), ocupan cuatro veces más espacio que un INT y hacen que las búsquedas sean significativamente más lentas.

Finalmente, la Llave Primaria Inteligente ofrece la ventaja de ser fácilmente identificable por humanos (como leer HER-TAL-001 y saber que es un taladro). No obstante, su desventaja es que es considerada una mala práctica, ya que viola la regla de oro: "los datos cambian, las llaves no". Si la categoría del producto cambia, la llave debe cambiar, lo que rompe la integridad referencial y obliga a realizar mantenimientos costosos y riesgosos en la base de datos.

¿Qué es una llave foránea?

En el diseño de bases de datos relacionales, una Llave Foránea (abreviada como FK) es un campo o conjunto de campos en una tabla que hace referencia a la Llave Primaria (PK) de otra tabla. Su función técnica es establecer un vínculo físico y lógico entre dos entidades, permitiendo que los datos "se hablen" entre sí sin necesidad de repetir toda la información. Por ejemplo, en el presente proyecto, la tabla de Ventas no necesita guardar el nombre del taladro, solo guarda el ID_Producto que apunta a la tabla maestra.

¿Cuáles son los tipos de llaves foráneas?

Para su implementación, la FK debe compartir obligatoriamente el mismo tipo de dato que la PK original (si el ID es INT, la foránea debe ser INT). Se define en el código SQL mediante la instrucción FOREIGN KEY (campo_local) REFERENCES tabla_maestra(campo_primario). Lo que realmente define la "personalidad" de la llave foránea es la regla de comportamiento que se le asigna:

- **Llave Foránea Simple (Simple Foreign Key)**

Es el tipo más común y directo que se utiliza al momento de realizar un sistema. Su uso es conectar una sola columna de la tabla "hija" con una sola columna de la tabla "padre". Su implementación es sencilla, ya que solo requiere que ambos campos tengan el mismo tipo de dato (como INT).

- **Llave Foránea Compuesta (Composite Foreign Key)**

Este tipo surge cuando la llave primaria a la que se desea apuntar no es un solo número, sino una combinación de varios campos. Su uso se da en sistemas complejos donde la identidad del "padre" depende de dos o más datos. Su implementación es más pesada, ya que en el SQL se debe declarar ambas columnas juntas como el vínculo de referencia.

- **Llave Foránea Recursiva (Recursive Foreign Key)**

Es un tipo especial de llave donde la columna hace referencia a la llave primaria de la misma tabla. Su uso es fundamental para crear jerarquías o estructuras de "árbol". Un ejemplo de ello sería que poseamos una tabla de Empleados; cada empleado tiene un id_empleado (PK), pero también queremos saber quién es su jefe. Creas una llave foránea llamada id_jefe que apunta al id_empleado de esa misma tabla. Así, el sistema entiende que un empleado reporta a otro dentro de la misma lista. Su implementación requiere cuidado para no crear ciclos infinitos (que un jefe se reporte a sí mismo).

- **Llave Foránea de Referencia Circular (Circular Foreign Key)**

Este es un caso avanzado donde dos tablas se apuntan mutuamente. Su uso ocurre cuando hay una dependencia recíproca. Por ejemplo, una tabla de Departamentos tiene una FK que apunta al id_empleado del gerente, y la tabla de Empleados tiene una FK que apunta al id_departamento donde trabaja. Su implementación es difícil de manejar porque al insertar el primer dato, el segundo aún no existe, por lo que se deben usar valores nulos temporales o desactivar las restricciones durante la carga inicial.

¿Cuáles son las ventajas y desventajas de los tipos de llaves foráneas?

Es vital entender que cada una afecta de forma distinta el rendimiento de un servidor y la complejidad del código.

- **Llave Foránea Simple (Simple Foreign Key)**

Es la conexión de una sola columna a una sola columna de otra tabla.

Ventajas: Es la más eficiente en términos de rendimiento. Al ser generalmente un número entero (INT), el motor de la base de datos realiza las búsquedas y las uniones (JOINs) de forma casi instantánea. Además, hace que el código SQL sea muy limpio y fácil de leer.

Desventajas: Su limitación es que no puede representar identidades complejas. Si el objeto que se desea relacionar depende de varios factores para ser único, una llave simple no será suficiente y será obligación crear una columna artificial extra que tal vez no se necesite.

- **Llave Foránea Compuesta (Composite Foreign Key)**

Es la que utiliza dos o más columnas para hacer referencia a una llave primaria compuesta en otra tabla.

Ventajas: Su mayor beneficio es que mantiene la lógica del mundo real sin necesidad de inventar IDs nuevos. Si un producto en la bodega se identifica por "Pasillo" y "Estante", la llave compuesta asegura que la relación sea exacta y natural, garantizando que no haya errores de ubicación en los pedidos.

Desventajas: A nivel técnico, es más pesada y lenta. El procesador debe comparar varios valores a la vez para validar la relación, lo que consume más CPU. Además, hace que las consultas JOIN sean mucho más largas y difíciles de escribir, aumentando las probabilidades de cometer errores de sintaxis en el código.

- **Llave Foránea Recursiva (Recursive Foreign Key)**

Es la que apunta a la llave primaria de su propia tabla para crear jerarquías.

Ventajas: Es la herramienta perfecta para crear estructuras organizativas (como jefes y empleados) o categorías con subcategorías (Pinturas -> Pinturas de Aceite -> Pinturas de Aceite Brillantes). Permite que una sola tabla maneje niveles infinitos de profundidad, lo que ahorra tener que crear múltiples tablas para lo mismo.

Desventajas: El riesgo técnico es la complejidad de las consultas y los bucles infinitos. Si no se programa con cuidado, es posible crear un error lógico donde el Empleado A es jefe del Empleado B, y el B es jefe del A, haciendo que el sistema colapse al intentar procesar la jerarquía. Además, borrar registros es más difícil porque se debe asegurar que no deje a toda una rama de la jerarquía sin "padre".

- **Llave Foránea de Referencia Circular (Circular Foreign Key)**

Es cuando dos tablas dependen mutuamente una de la otra.

Ventajas: Permite modelar dependencias recíprocas muy estrictas, como asegurar que todo departamento tenga un gerente y que todo gerente pertenezca a un departamento obligatoriamente.

Desventajas: Es una de las estructuras más difíciles de implementar. El problema principal ocurre al insertar datos: no es posible insertar al gerente porque el departamento no existe, y no es posible insertar el departamento porque el gerente no existe. Esto nos obliga a usar trucos técnicos (como permitir valores nulos temporales), lo que debilita un poco la seguridad inicial de los datos.

¿Qué es la normalización?

La normalización es un conjunto de reglas que se aplican al diseño de tablas para evitar anomalías al insertar, actualizar o borrar registros. Se divide en niveles llamados Formas Normales (FN). Para que una base de datos sea profesional, debe cumplir al menos las tres primeras

¿Cuáles son las formas normales que existen?

La normalización es un proceso acumulativo: para que una tabla esté en una forma normal superior, primero debe cumplir estrictamente con todas las anteriores.

- **Primera Forma Normal (1FN): Atomicidad**

Es el nivel más básico. Su uso es eliminar grupos de datos repetidos y asegurar que cada columna contenga un solo valor indivisible. Por ejemplo, en un sistema, no podemos tener una columna "Productos" con una lista de 5 herramientas; debemos tener una fila por cada herramienta. Su implementación requiere que cada tabla tenga una Llave Primaria y que no existan atributos con múltiples valores en una sola celda.

- **Segunda Forma Normal (2FN): Dependencia Completa**

Su uso es eliminar la redundancia cuando usamos llaves primarias compuestas. Establece que todos los datos de la tabla deben depender de toda la llave primaria, no solo de una parte. Si se tiene una tabla de Detalle_Venta (ID_Venta + ID_Producto), el "Precio_Unitario" depende del producto, pero el "Nombre_Marca" solo depende del producto y no de la venta, por lo que debe salir a su propia tabla de Productos.

- **Tercera Forma Normal (3FN): Independencia Transitiva**

Es el estándar de la industria. Su uso es asegurar que los datos dependan únicamente de la llave primaria y no de otros campos no clave. Si en una tabla de Clientes se posee el "Código Postal" y la "Ciudad", la ciudad depende del código postal, no del cliente. Su implementación consiste en crear una tabla de Localidades para separar esa relación.

- **Forma Normal de Boyce-Codd (BCNF)**

Es una versión más estricta de la 3FN, a veces llamada "3.5 FN". Su uso ocurre en casos complejos donde existen varias llaves candidatas que se solapan (tienen columnas en común). Su implementación asegura que cada determinante en la tabla sea una llave candidata, eliminando anomalías que la 3FN simple no alcanza a cubrir.

- **Cuarta Forma Normal (4FN): Dependencias Multivaluadas**

Su uso es para tablas que intentan guardar dos o más relaciones independientes de "muchos a muchos". Por ejemplo, si un Proveedor vende muchos Tipos_de_Pintura y también tiene muchas Sucursales, no deberías mezclarlos en una sola tabla de proveedores, ya que se crearían combinaciones innecesarias de datos. Su implementación requiere separar esas relaciones independientes en dos tablas distintas.

- **Quinta Forma Normal (5FN): Forma Normal de Proyección-Unión**

También conocida como PJ/NF. Su uso es extremadamente raro y se aplica cuando una tabla no puede ser reconstruida a partir de tablas más pequeñas sin perder información, a menos que se divida en tres o más partes. Su implementación garantiza que no existan anomalías de unión lógica en estructuras de datos muy complejas.

- **Sexta Forma Normal (6FN): Normalización Temporal**

Es el nivel más alto y se utiliza principalmente en bases de datos temporales. Su uso es descomponer las tablas hasta que solo contengan la llave primaria y un solo atributo adicional que varía con el tiempo. Su implementación permite llevar un registro histórico perfecto de cada cambio.

¿Cuáles son las ventajas y desventajas de la normalización?

Es fundamental entender que la normalización no es solo "ordenar por ordenar". Es un intercambio técnico: ganas orden y seguridad, pero a veces sacrificas un poco de velocidad de lectura.

Ventajas de la Normalización

- **Eliminación de la Redundancia:** Es su mayor beneficio. Al no repetir el nombre del proveedor o la marca del producto en cada venta, ahorrando espacio físico en el disco duro del servidor. En sistemas grandes, esto reduce el peso de la base de datos hasta en un 60%.
- **Integridad de los Datos:** Evita las "anomalías de actualización". Si una categoría cambia de nombre, solo se edita en una sola celda de la tabla de categorías, y el cambio se refleja instantáneamente en los miles de productos asociados. No hay riesgo de que un dato quede desactualizado.
- **Facilidad de Mantenimiento:** Al tener tablas pequeñas y específicas (una para Clientes, otra para Productos, otra para Ventas), el código del sistema es mucho más fácil de entender y ampliar. Si se desea agregar una nueva característica a los productos, solo sería necesario editar la tabla de Productos sin arriesgarse a romper la lógica de las Ventas.
- **Optimización de Índices:** Las tablas normalizadas suelen ser "más delgadas" (menos columnas). Esto permite que el motor de la base de datos cree índices más eficientes en la memoria RAM, haciendo que las búsquedas por ID sean extremadamente veloces.

Desventajas de la Normalización

- **Complejidad en las Consultas (JOINs):** Esta es la desventaja más notable para un programador. Al tener los datos repartidos en 10 tablas en lugar de una, para mostrar una factura completa en la web será necesario usar múltiples JOINs en SQL. Esto hace que el código sea más largo y complejo de escribir.
- **Impacto en el Rendimiento de Lectura:** Cada vez que el servidor debe "unir" tablas para mostrar una información, consume ciclos de CPU y memoria RAM. En sistemas con millones de usuarios simultáneos, unir 5 o 6 tablas puede ralentizar la carga de la página un par de milisegundos más que si la tabla fuera una sola "tabla plana".
- **Dificultad de Comprensión Inicial:** Para alguien que no sabe de bases de datos, ver un modelo normalizado puede ser confuso, ya que verá muchos números (IDs) en lugar de nombres directos. Requiere que el equipo técnico tenga un Diagrama Entidad-Relación actualizado para no perderse.
- **Sobrenormalización:** Existe el riesgo de llevar la normalización al extremo (4FN, 5FN o 6FN) en proyectos sencillos. Esto crea una fragmentación excesiva de los d atos que no aporta beneficios reales y solo complica innecesariamente la programación del sistema.

¿Qué es un SGBD (Sistema de Gestión de Bases de Datos)?

Un SGBD (en inglés DBMS, Database Management System) es un software especializado que actúa como una capa intermedia entre los datos físicos almacenados en el disco duro y los usuarios o aplicaciones. Su función técnica principal es administrar, recuperar y proteger la información de manera eficiente. Se podría decir que la base de datos es una gran bodega llena de estantes, mientras que el SGBD es el gerente experto que posee las llaves, conoce la ubicación exacta de cada objeto y garantiza que nadie entre sin permiso o dañe la mercancía. Sin un SGBD como MySQL o Microsoft SQL Server, se debería de escribir código manual extremadamente complejo y lento solo para leer un simple dato, lo que aumentaría drásticamente el riesgo de errores y pérdida de información.

En cuanto a su uso, el SGBD es el responsable de ejecutar las cuatro operaciones fundamentales conocidas como CRUD (Crear, Leer, Actualizar y Borrar). Además, implementa las Propiedades ACID (Atomicidad, Consistencia, Aislamiento y Durabilidad), que son las reglas de oro para que las transacciones sean seguras. Por ejemplo, si un cliente está pagando una pieza de PC y se va la luz, el SGBD asegura que el dinero no se descuente si la orden no se completó, devolviendo todo a su estado original. También permite el acceso multiusuario, logrando que cientos de clientes consulten el stock del sistema al mismo tiempo sin que los datos se bloqueen o se corrompan.

Para su implementación, el SGBD utiliza un lenguaje estándar llamado SQL (Structured Query Language). Internamente, el sistema cuenta con un motor de consultas que optimiza las búsquedas para que sean instantáneas, un Gestor de Memoria que utiliza la RAM para acelerar los procesos y un Log de Transacciones que anota cada movimiento antes de realizarlo para poder recuperarse de fallos técnicos. En el mercado existen diferentes opciones: MySQL es el líder en desarrollo web por ser gratuito y ligero, mientras que Microsoft SQL Server es una solución empresarial robusta diseñada para manejar volúmenes masivos de datos con herramientas de análisis avanzadas.

¿Cuáles son los tipos de SGBD?

En el mundo de la infraestructura tecnológica, un SGBD es el motor de software que define las reglas de almacenamiento, seguridad y recuperación de la información. Para que no haya duda al respecto, estos se clasifican en cuatro grandes familias según su arquitectura y el problema específico que resuelven en la industria:

- **SGBD Relacionales (RDBMS)**

Es el estándar de oro de la informática. Su Estructura organiza la información en tablas (relaciones) compuestas por filas y columnas, basándose firmemente en el álgebra relacional y utilizando el lenguaje estándar SQL para todas sus operaciones de consulta. En cuanto al Detalle Técnico, su mayor fuerza es el cumplimiento estricto de las Propiedades ACID (Atomicidad, Consistencia, Aislamiento y Durabilidad), lo que lo convierte en el modelo ideal para datos estructurados donde la relación entre tablas

mediante llaves primarias y foráneas debe ser perfecta para garantizar la integridad total. Su Uso principal se encuentra en sistemas de inventario de hardware, facturación y registros de usuarios con alta seguridad, contando con ejemplos líderes como MySQL, PostgreSQL, MariaDB, Microsoft SQL Server y Oracle.

- **SGBD No Relacionales (NoSQL)**

Surgieron para romper la rigidez de las tablas tradicionales cuando se necesita procesar volúmenes masivos de datos desordenados, cambiantes o que requieren una velocidad de respuesta extrema en tiempo real. Su Estructura tiene un diseño dinámico y no utiliza esquemas fijos, permitiendo almacenar la información en documentos (tipo JSON), pares de clave-valor, columnas anchas o grafos, lo que facilita que cada registro tenga campos diferentes si es necesario. En el Detalle Técnico, su principal virtud es la escalabilidad horizontal, lo que permite distribuir los datos en cientos de servidores para manejar millones de peticiones por segundo, aunque a menudo sacrifican la consistencia perfecta por una velocidad superior en la lectura y escritura. Su Uso es fundamental en redes sociales (perfiles y muros), análisis de Big Data, carritos de compra temporales y aplicaciones de mensajería masiva, destacando ejemplos como MongoDB, Redis, Cassandra, Neo4j y Amazon DynamoDB.

- **SGBD Orientados a Objetos (ODBMS)**

Estos sistemas están diseñados para integrarse de forma nativa con el flujo de trabajo de los programadores de software avanzado, eliminando la barrera entre el código y el almacenamiento. Su Estructura almacena la información directamente en forma de objetos, tal como se definen en lenguajes como Java, C++ o Python, incluyendo tanto sus datos (atributos) como sus comportamientos (métodos). Su mayor ventaja es eliminar la "desconexión de impedancia", que es el esfuerzo computacional de traducir tablas de SQL a objetos de código, lo que agiliza drásticamente el rendimiento en aplicaciones donde los datos tienen jerarquías muy profundas o complejas. Su Uso se especializa en sistemas de diseño asistido por computadora (CAD), simulaciones científicas de alta precisión y bases de datos multimedia, contando con ejemplos como ObjectDB, db4o, Zope Object Database y Perst.

- **SGBD Distribuidos**

Representan la solución de ingeniería definitiva para sistemas globales que requieren una resistencia total a fallos y una alta disponibilidad geográfica. Su Estructura consiste en una base de datos que no reside en una sola computadora, sino que la información está físicamente repartida en múltiples servidores conectados por una red, aunque para el usuario final funcione como una sola unidad lógica centralizada. El sistema gestiona procesos críticos de fragmentación y replicación, asegurando que si un servidor en una ciudad o país falla, los datos se recuperen automáticamente de otro nodo, garantizando que el servicio nunca se interrumpa. Su Uso es la columna vertebral de empresas globales como Google, Facebook, sistemas de reservas de vuelos internacionales y transacciones bancarias transnacionales.

¿Cuáles son sus ventajas y desventajas de los SGBD?

- **SGBD Relacionales (RDBMS)**

Ventajas: Su mayor beneficio es la integridad y consistencia absoluta de los datos; gracias al cumplimiento de las propiedades ACID, es casi imposible que una transacción quede a medias o que existan datos corruptos. Además, al usar el estándar SQL, es muy fácil encontrar personal capacitado o herramientas compatibles. Su estructura normalizada reduce al mínimo la redundancia, ahorrando espacio en disco y facilitando el mantenimiento a largo plazo.

Desventajas: Su principal debilidad es la dificultad para escalar horizontalmente; para manejar más tráfico, generalmente es necesario un servidor más potente (escalabilidad vertical), lo cual es costoso. Además, cuando las tablas crecen a miles de millones de registros, los JOINs complejos pueden volverse lentos, y su esquema rígido obliga a definir toda la estructura antes de insertar el primer dato, lo que quita agilidad en proyectos que cambian rápido.

- **SGBD No Relacionales (NoSQL)**

Ventajas: Ofrecen una escalabilidad y flexibilidad incomparables; es posible añadir servidores económicos en cadena para manejar peticiones masivas sin detener el sistema. Al no tener un esquema fijo, permiten que los desarrolladores guarden datos con diferentes estructuras sobre la marcha, lo que acelera drásticamente el tiempo de desarrollo en aplicaciones modernas. Son extremadamente rápidos para operaciones de lectura y escritura simples en grandes volúmenes de datos.

Desventajas: El gran sacrificio es la consistencia eventual; en sistemas muy grandes, es posible que un usuario lea un dato un milisegundo antes de que se actualice en todos los nodos. Además, carecen de un lenguaje estándar universal como SQL, lo que obliga a aprender una sintaxis diferente para cada base de datos (como la de MongoDB o Cassandra). Tampoco son ideales para aplicaciones donde las relaciones complejas entre datos son la prioridad.

- **SGBD Orientados a Objetos (ODBMS)**

Ventajas: La principal ventaja es el alto rendimiento en la manipulación de objetos complejos; al eliminar la necesidad de traducir tablas a código (mapeo objeto-relacional), el acceso a los datos es directo y mucho más veloz para aplicaciones de ingeniería o multimedia. Esto reduce la cantidad de código que el programador debe escribir y mantener, logrando una integración perfecta con lenguajes como Java o C++.

Desventajas: Tienen una comunidad y soporte mucho más limitados en comparación con los sistemas relacionales. Es difícil encontrar herramientas de reportes o análisis de datos externas que se conecten de forma nativa a ellos. Además, su aprendizaje es más especializado y no son la mejor opción para aplicaciones administrativas simples donde una tabla de SQL resolvería el problema de forma más barata y estándar.

- **SGBD Distribuidos**

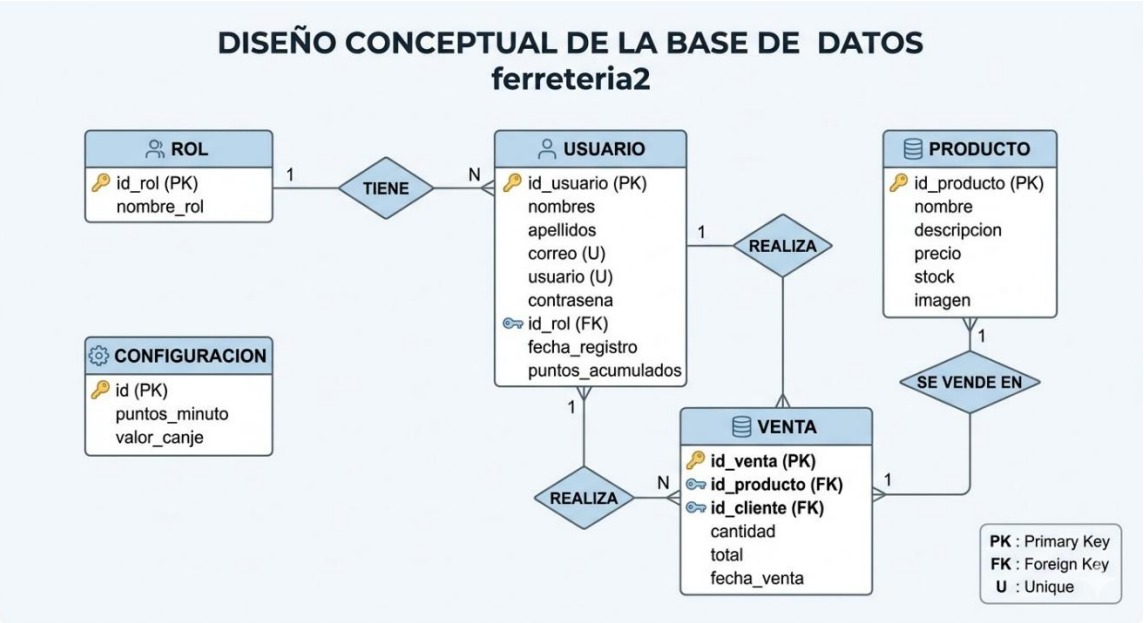
Ventajas: Su punto más fuerte es la alta disponibilidad y tolerancia a fallos; el sistema está diseñado para que, si un centro de datos entero se inunda o se queda sin energía, los otros nodos asuman la carga instantáneamente sin que el usuario lo note. También permiten que los datos estén físicamente cerca del usuario final (por ejemplo, datos

en servidores de El Salvador para usuarios locales), reduciendo la latencia y mejorando la experiencia de uso global.

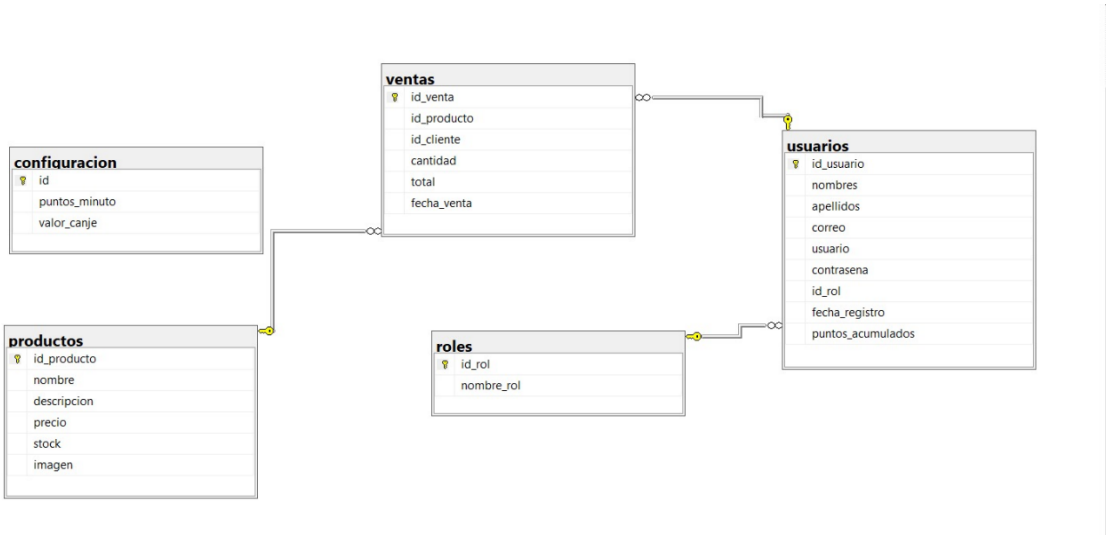
Desventajas: La extrema complejidad de administración es su mayor contra; sincronizar los datos para que todos los servidores tengan la misma información al mismo tiempo es un desafío de ingeniería muy costoso. Requieren una infraestructura de red impecable y personal altamente especializado para configurarlos, lo que eleva drásticamente los costos operativos en comparación con una base de datos centralizada.

Diseño de la Base de Datos

Diseño Conceptual



Diseño Lógico



Diseño Físico

```sql
CREATE DATABASE ferreteria2;
use ferreteria2;

CREATE TABLE roles (
    id_rol INT(11) NOT NULL AUTO_INCREMENT,
    nombre_rol VARCHAR(50) NOT NULL,
    PRIMARY KEY (id_rol)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

CREATE TABLE configuracion (
    id INT(11) NOT NULL AUTO_INCREMENT,
    puntos_minuto INT(11) DEFAULT 100,
    valor_canje DECIMAL(10,2) DEFAULT 100.00,
    PRIMARY KEY (id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

```sql
    PRIMARY KEY (id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

CREATE TABLE usuarios (
    id_usuario INT(11) NOT NULL AUTO_INCREMENT,
    nombres VARCHAR(100) NOT NULL,
    apellidos VARCHAR(100) NOT NULL,
    correo VARCHAR(100) NOT NULL,
    usuario VARCHAR(50) NOT NULL,
    contrasena VARCHAR(255) NOT NULL,
    id_rol INT(11),
    fecha_registro DATETIME DEFAULT CURRENT_TIMESTAMP,
    puntos_acumulados DECIMAL(10,2) DEFAULT 0.00,
    PRIMARY KEY (id_usuario),
    UNIQUE KEY (correo),
    UNIQUE KEY (usuario),
```

```
    PRIMARY KEY (id_usuario),
    UNIQUE KEY (correo),
    UNIQUE KEY (usuario),
    CONSTRAINT fk_usuario_rol FOREIGN KEY (id_rol) REFERENCES roles(id_rol) ON DELETE SET NULL ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

CREATE TABLE productos (
    id_producto INT(11) NOT NULL AUTO_INCREMENT,
    nombre VARCHAR(100) NOT NULL,
    descripcion TEXT,
    precio DECIMAL(10,2) NOT NULL,
    stock INT(11) NOT NULL DEFAULT 0,
    imagen VARCHAR(255),
    PRIMARY KEY (id_producto)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

```
    PRIMARY KEY (id_producto)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

CREATE TABLE ventas (
    id_venta INT(11) NOT NULL AUTO_INCREMENT,
    id_producto INT(11) NOT NULL,
    id_cliente INT(11),
    cantidad INT(11) NOT NULL,
    total DECIMAL(10,2) NOT NULL,
    fecha_venta DATETIME DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (id_venta),
    CONSTRAINT fk_venta_producto FOREIGN KEY (id_producto) REFERENCES productos(id_producto) ON DELETE CASCADE ON UPDATE
CASCADE,
    CONSTRAINT fk_venta_cliente FOREIGN KEY (id_cliente) REFERENCES usuarios(id_usuario) ON DELETE SET NULL ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

```
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- vistas
-- Vista para ver el inventario crítico
CREATE OR REPLACE VIEW vista_stock_bajo AS
SELECT id_producto, nombre, stock, precio
FROM productos
WHERE stock < 10;

-- Vista para ver detalles de ventas con nombres de clientes y productos
CREATE OR REPLACE VIEW vista_ventas_detalladas AS
SELECT
    v.id_venta,
    u.usuario AS cliente,
    p.nombre AS producto,
    v.cantidad,
    v.total,
    v.fecha_venta
FROM ventas v
LEFT JOIN usuarios u ON v.id_cliente = u.id_usuario
INNER JOIN productos p ON v.id_producto = p.id_producto;

-- procedimiento almacenado
DELIMITER //
```

```
-- procedimiento almacenado
DELIMITER //

-- Procedimiento para registrar una venta y descontar stock automáticamente
CREATE PROCEDURE sp_registrar_venta(
    IN p_id_producto INT,
    IN p_id_cliente INT,
    IN p_cantidad INT
)
BEGIN
    DECLARE v_precio DECIMAL(10,2);
    DECLARE v_total DECIMAL(10,2);

    -- Obtener el precio del producto
    SELECT precio INTO v_precio FROM productos WHERE id_producto = p_id_producto;

    -- Calcular el total
    SET v_total = v_precio * p_cantidad;

    -- Insertar la venta
    INSERT INTO ventas (id_producto, id_cliente, cantidad, total)
    VALUES (p_id_producto, p_id_cliente, p_cantidad, v_total);

    -- Actualizar el stock del producto
    UPDATE productos SET stock = stock - p_cantidad WHERE id_producto = p_id_producto;
```

```
BEGIN
    DECLARE v_precio DECIMAL(10,2);
    DECLARE v_total DECIMAL(10,2);

    -- Obtener el precio del producto
    SELECT precio INTO v_precio FROM productos WHERE id_producto = p_id_producto;

    -- Calcular el total
    SET v_total = v_precio * p_cantidad;

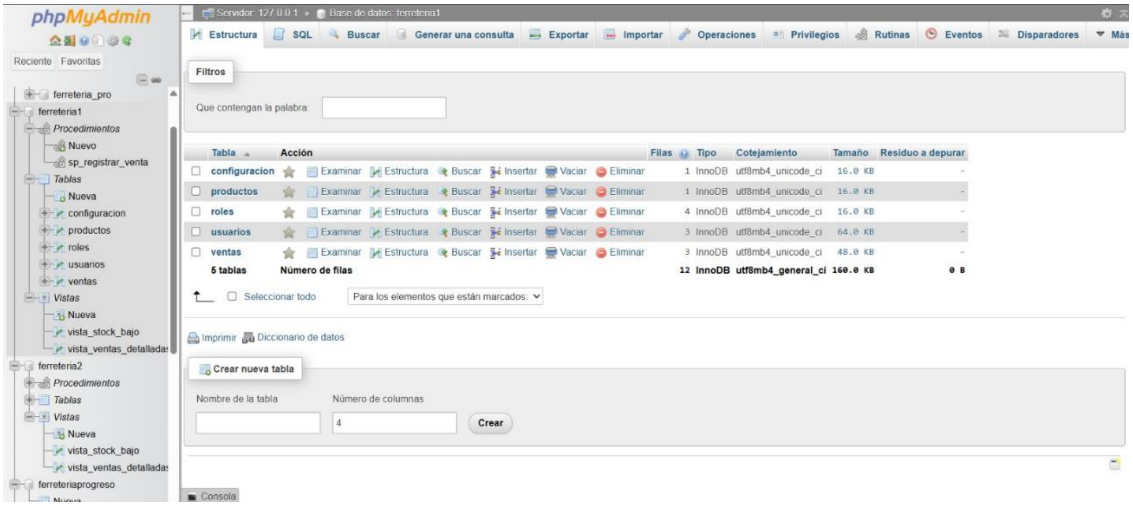
    -- Insertar la venta
    INSERT INTO ventas (id_producto, id_cliente, cantidad, total)
    VALUES (p_id_producto, p_id_cliente, p_cantidad, v_total);

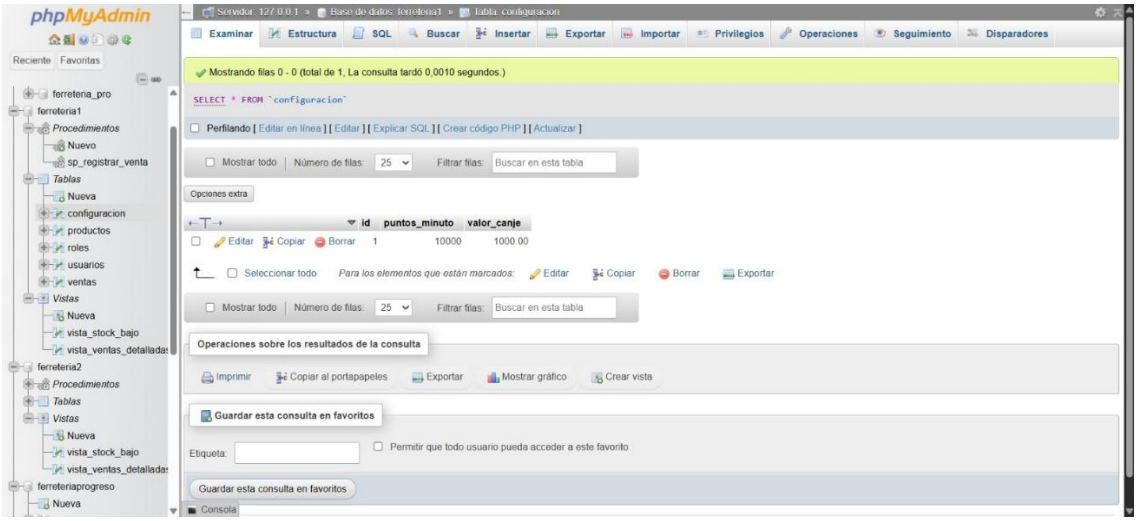
    -- Actualizar el stock del producto
    UPDATE productos SET stock = stock - p_cantidad WHERE id_producto = p_id_producto;
END //

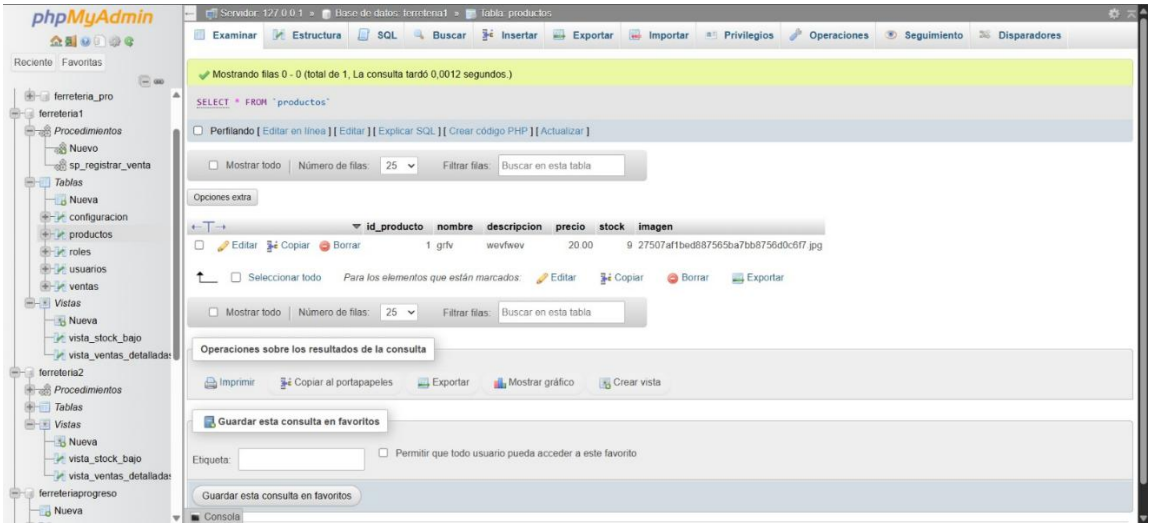
DELIMITER ;

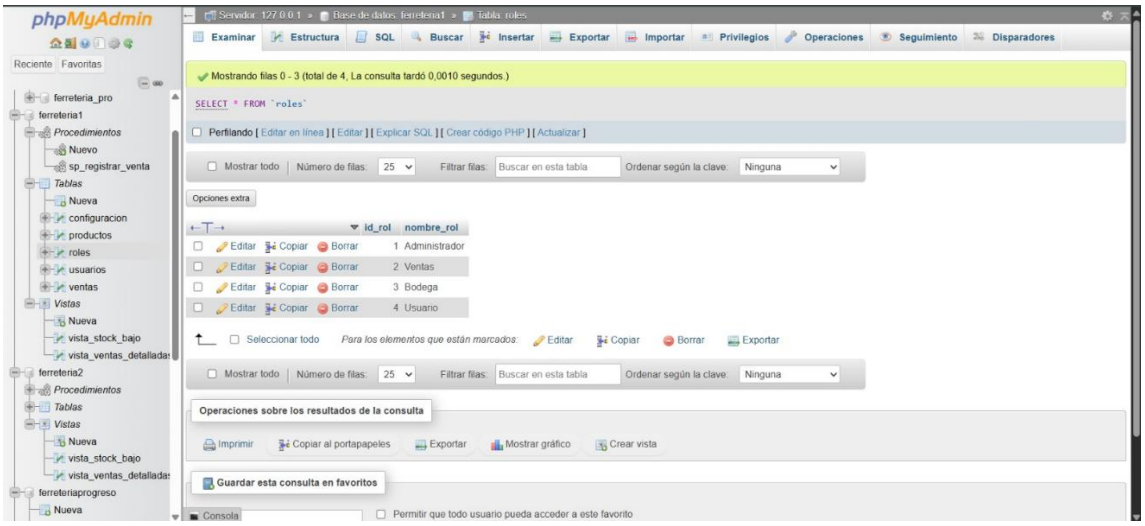
INSERT INTO roles (id_rol, nombre_rol) VALUES (1, 'Administrador'), (2, 'Ventas'), (3, 'Bodega'), (4, 'Usuario');
INSERT INTO configuracion (id, puntos_minuto, valor_canje) VALUES (1, 100, 100.00);
```

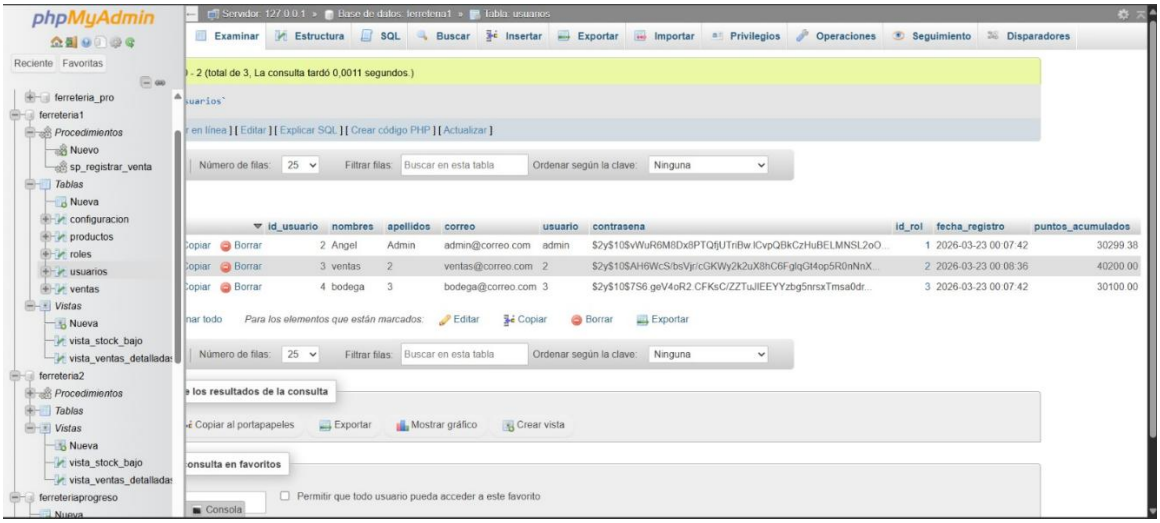
Desarrollo del Proyecto

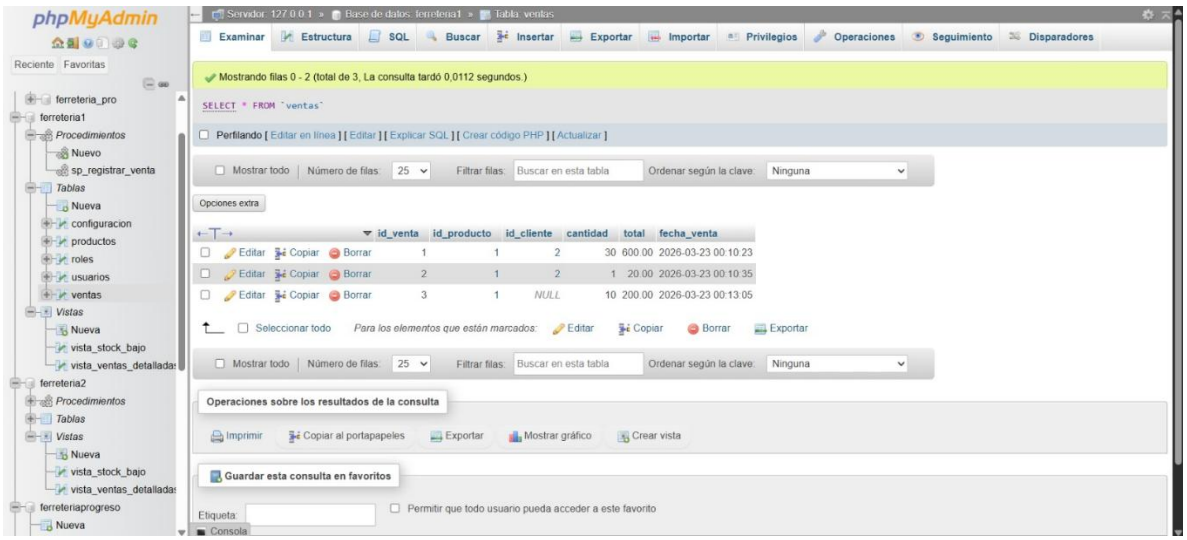


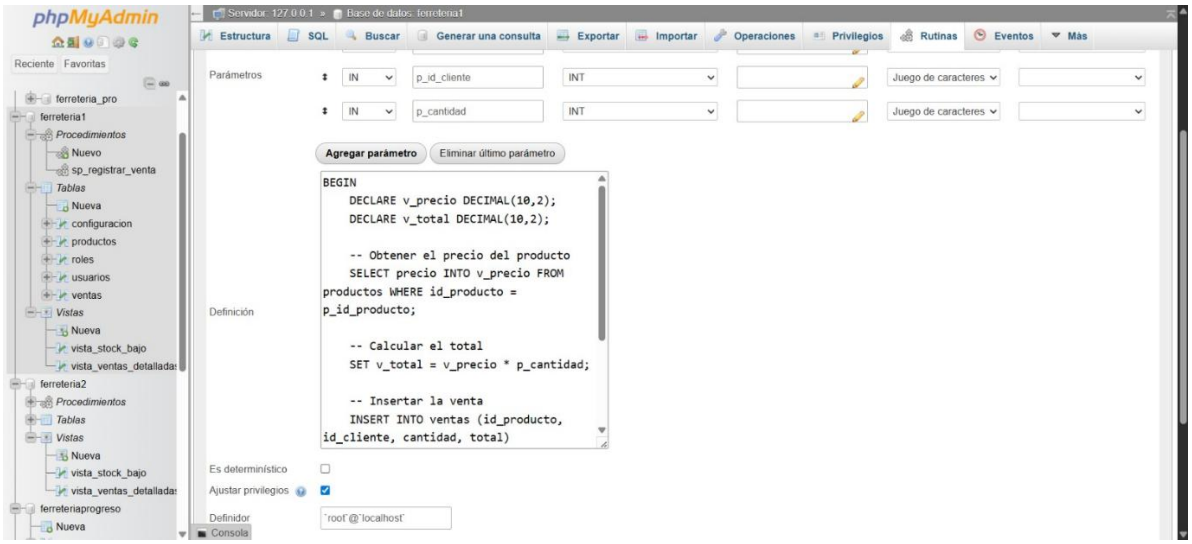


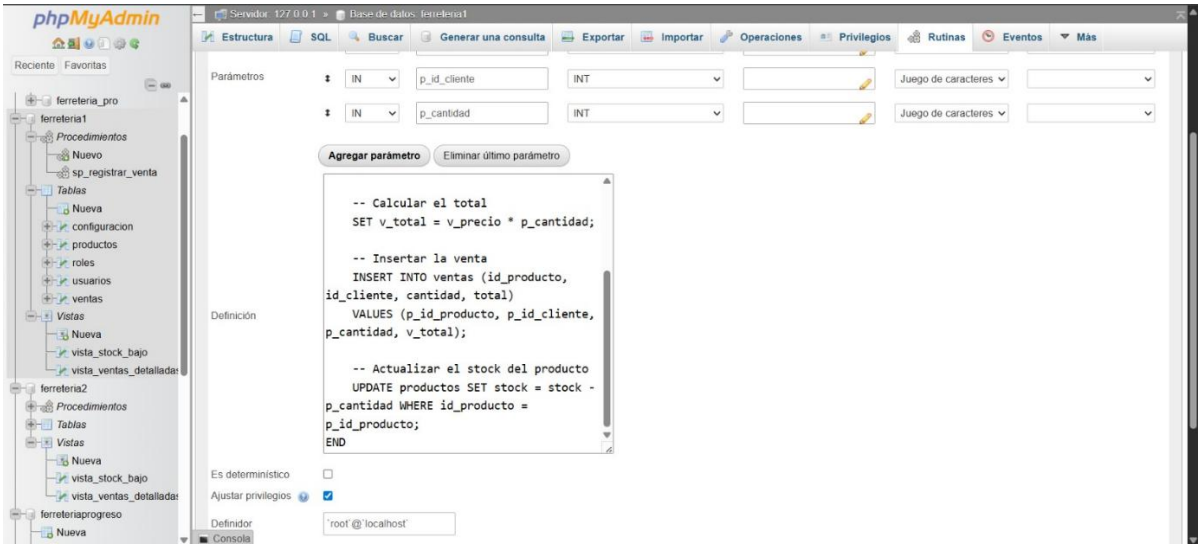


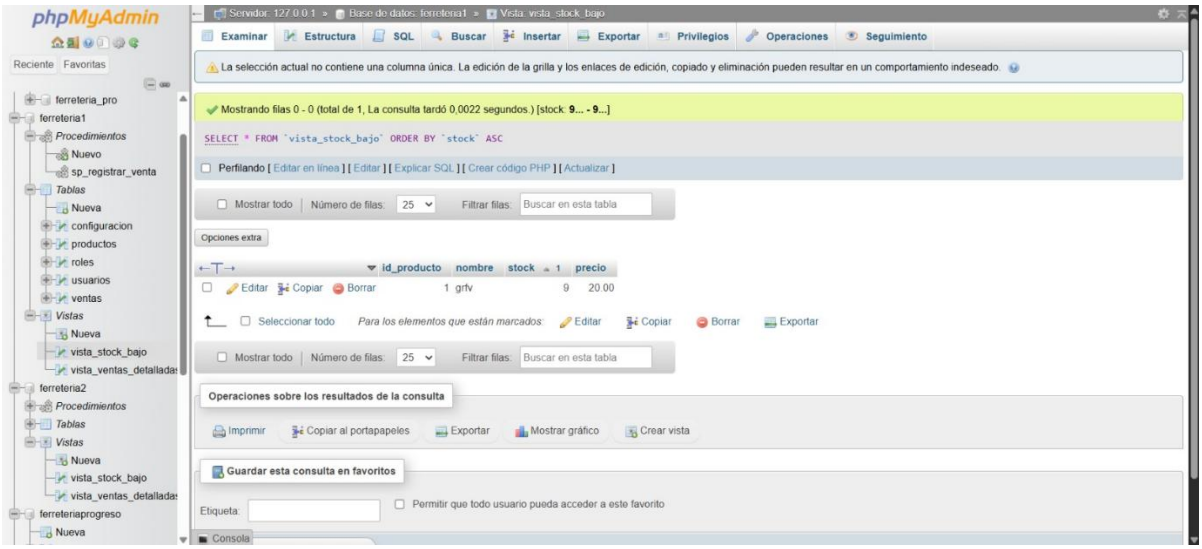

phpMyAdmin
Servidor: 127.0.0.1 » Base de datos: ferreteria1 » Tabla: roles
Examinar Estructura SQL Buscar Insertar Exportar Importar Privilegios Operaciones Seguimiento Disparadores
Mostrando filas 0 - 3 (total de 4, La consulta tardó 0,0010 segundos.)
SELECT * FROM `roles`
Perfilando [Editar en línea] [Editar] [Explicar SQL] [Crear código PHP] [Actualizar]
Mostrar todo | Número de filas: 25 | Filtrar filas: Buscar en esta tabla | Ordenar según la clave: Ninguna
Opciones extra
id_rol nombre_rol
Editar Copiar Borrar 1 Administrador
Editar Copiar Borrar 2 Ventas
Editar Copiar Borrar 3 Bodega
Editar Copiar Borrar 4 Usuario
Seleccionar todo Para los elementos que están marcados: Editar Copiar Borrar Exportar
Operaciones sobre los resultados de la consulta
Imprimir Copiar al portapapeles Exportar Mostrar gráfico Crear vista
Guardar esta consulta en favoritos
Permitir que todo usuario pueda acceder a este favorito
Consola

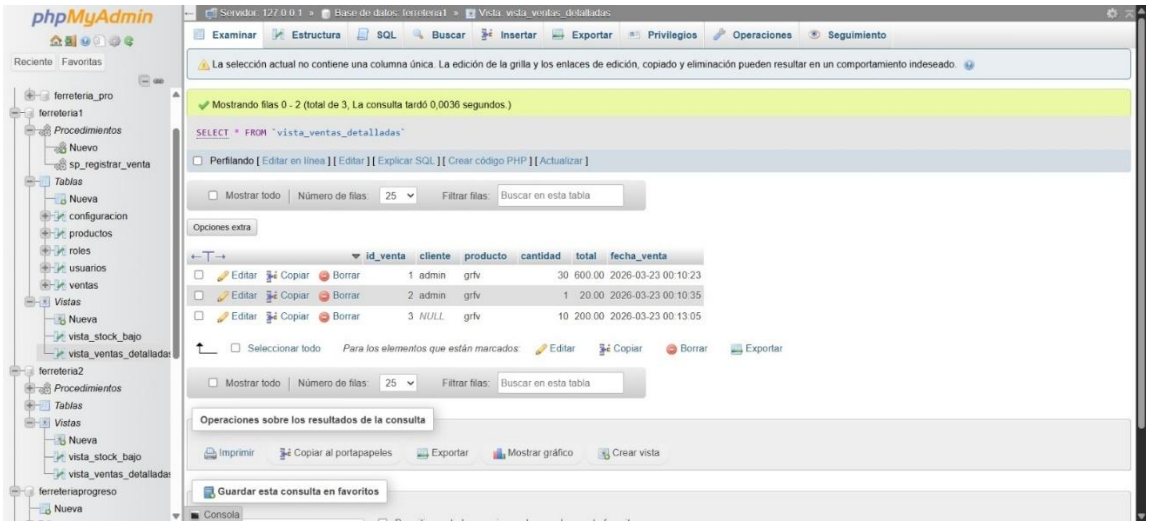

phpMyAdmin
Servidor: 127.0.0.1 » Base de datos: ferreteria1 » Tabla: usuarios
Examinar Estructura SQL Buscar Insertar Exportar Importar Privilegios Operaciones Seguimiento Disparadores
id_usuario nombres apellidos correo usuario contrasena id_rol fecha_registro puntos_acumulados
Copiar Borrar 2 Angel Admin admin@correo.com admin $2y$10$vWuR6M8Dx8PTQfjUTnBw.ICvpQBkCzHuBELMNSL2oO... 1 2026-03-23 00:07:42 30299.38
Copiar Borrar 3 ventas 2 ventas@correo.com 2 $2y$10$AH6WcS/bsVjr/cGKWy2k2uX8hC6FglqGt4op5R0nNnX... 2 2026-03-23 00:08:36 40200.00
Copiar Borrar 4 bodega 3 bodega@correo.com 3 $2y$10$7S6.geV4oR2.CFKsC/ZZTuJlEEYYzbg5nrsxTmsa0dr... 3 2026-03-23 00:07:42 30100.00
Para los elementos que están marcados: Editar Copiar Borrar Exportar
Copiar al portapapeles Exportar Mostrar gráfico Crear vista
Permitir que todo usuario pueda acceder a este favorito
Consola

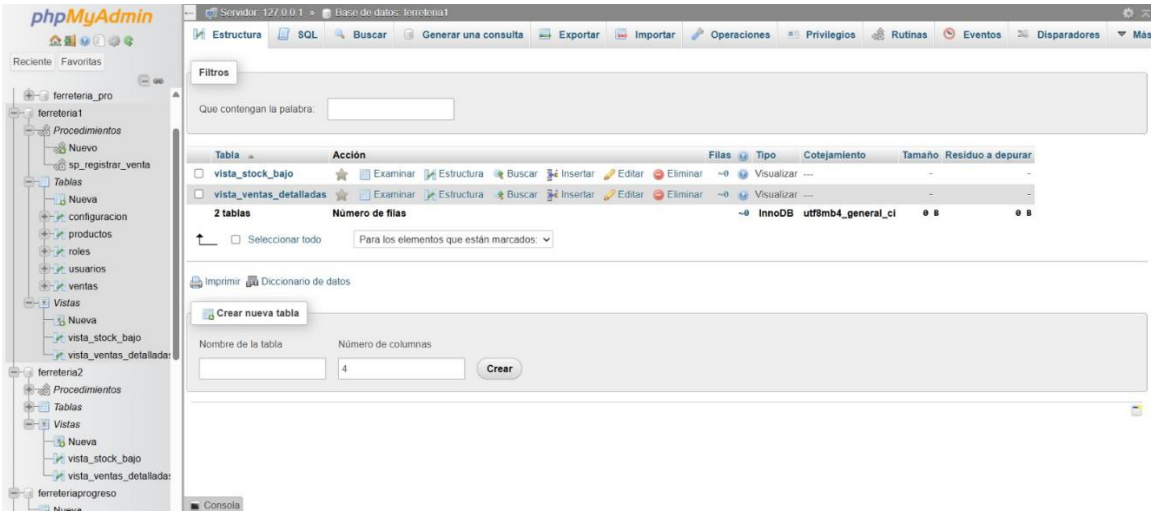

phpMyAdmin
Servidor: 127.0.0.1 » Base de datos: ferreteria1 » Tabla: ventas
Examinar Estructura SQL Buscar Insertar Exportar Importar Privilegios Operaciones Seguimiento Disparadores
Mostrando filas 0 - 2 (total de 3, La consulta tardó 0,0112 segundos.)
SELECT * FROM `ventas`
Perfilando [Editar en línea] [Editar] [Explicar SQL] [Crear código PHP] [Actualizar]
Mostrar todo | Número de filas: 25 | Filtrar filas: Buscar en esta tabla | Ordenar según la clave: Ninguna
Opciones extra
id_venta id_producto id_cliente cantidad total fecha_venta
Editar Copiar Borrar 1 1 2 30 600.00 2026-03-23 00:10:23
Editar Copiar Borrar 2 1 2 1 20.00 2026-03-23 00:10:35
Editar Copiar Borrar 3 1 NULL 10 200.00 2026-03-23 00:13:05
Seleccionar todo Para los elementos que están marcados: Editar Copiar Borrar Exportar
Operaciones sobre los resultados de la consulta
Imprimir Copiar al portapapeles Exportar Mostrar gráfico Crear vista
Guardar esta consulta en favoritos
Etiqueta:
Permitir que todo usuario pueda acceder a este favorito
Consola

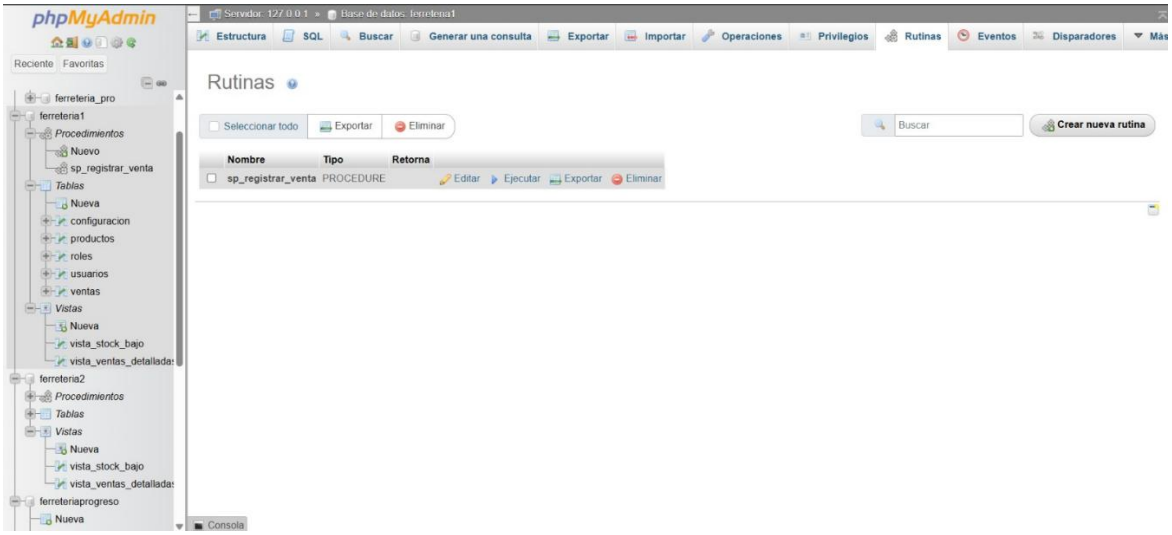

phpMyAdmin
Servidor: 127.0.0.1 » Base de datos: ferreteria1
Estructura SQL Buscar Generar una consulta Exportar Importar Operaciones Privilegios Rutinas Eventos Más
Parámetros
IN p_id_cliente INT Juego de caracteres
IN p_cantidad INT Juego de caracteres
Agregar parámetro Eliminar último parámetro
Definición
BEGIN
DECLARE v_precio DECIMAL(10,2);
DECLARE v_total DECIMAL(10,2);
-- Obtener el precio del producto
SELECT precio INTO v_precio FROM productos WHERE id_producto = p_id_producto;
-- Calcular el total
SET v_total = v_precio * p_cantidad;
-- Insertar la venta
INSERT INTO ventas (id_producto, id_cliente, cantidad, total)
Es determinístico
Ajustar privilegios
Definidor
`root`@`localhost`
Consola


phpMyAdmin
Servidor: 127.0.0.1 » Base de datos: ferreteria1
Estructura SQL Buscar Generar una consulta Exportar Importar Operaciones Privilegios Rutinas Eventos Más
Parámetros
IN p_id_cliente INT Juego de caracteres
IN p_cantidad INT Juego de caracteres
Agregar parámetro Eliminar último parámetro
Definición
-- Calcular el total
SET v_total = v_precio * p_cantidad;
-- Insertar la venta
INSERT INTO ventas (id_producto, id_cliente, cantidad, total)
VALUES (p_id_producto, p_id_cliente, p_cantidad, v_total);
-- Actualizar el stock del producto
UPDATE productos SET stock = stock - p_cantidad WHERE id_producto = p_id_producto;
END
Es determinístico
Ajustar privilegios
Definidor
`root`@`localhost`
Consola


phpMyAdmin
Servidor: 127.0.0.1 » Base de datos: ferreteria1 » Vista: vista_stock_bajo
Examinar Estructura SQL Buscar Insertar Exportar Privilegios Operaciones Seguimiento
La selección actual no contiene una columna única. La edición de la grilla y los enlaces de edición, copiado y eliminación pueden resultar en un comportamiento indeseado.
Mostrando filas 0 - 0 (total de 1, La consulta tardó 0,0022 segundos.) [stock: 9... - 9...]
SELECT * FROM `vista_stock_bajo` ORDER BY `stock` ASC
Perfilando [Editar en línea] [Editar] [Explicar SQL] [Crear código PHP] [Actualizar]
Mostrar todo Número de filas: 25 Filtrar filas: Buscar en esta tabla
Opciones extra
id_producto nombre stock 1 precio
Editar Copiar Borrar 1 grfv 9 20.00
Seleccionar todo Para los elementos que están marcados: Editar Copiar Borrar Exportar
Operaciones sobre los resultados de la consulta
Imprimir Copiar al portapapeles Exportar Mostrar gráfico Crear vista
Guardar esta consulta en favoritos
Etiqueta:
Permitir que todo usuario pueda acceder a este favorito
Consola


phpMyAdmin
Examinar
Estructura
SQL
Buscar
Insertar
Exportar
Privilegios
Operaciones
Seguimiento
Mostrando filas 0 - 2 (total de 3, La consulta tardó 0,0036 segundos.)
SELECT * FROM `vista_ventas_detalladas`
Operaciones sobre los resultados de la consulta
Guardar esta consulta en favoritos


phpMyAdmin
Estructura
SQL
Buscar
Generar una consulta
Exportar
Importar
Operaciones
Privilegios
Rutinas
Eventos
Disparadores
Filtros
Crear nueva tabla
Nombre de la tabla
Número de columnas
Crear


phpMyAdmin
Rutinas
Crear nueva rutina
sp_registrar_venta
PROCEDURE

registro.php

```
['REQUEST_METHOD'] === 'POST') {
y($errores)) {
lyExiste = "SELECT id_usuario FROM usuarios WHERE correo = '$correo' OR usuario = '$usuario'";
Existe = mysqli_query($db, $queryExiste);

mysqli_num_rows($resExiste) > 0) {
$errores[] = "El correo o el nombre de usuario ya están registrados";
se {
// HASHEAR la contraseña antes de guardarla
$passwordHash = password_hash($password, PASSWORD_BCRYPT);

// insertar con todos los campos necesarios
$query = "INSERT INTO usuarios (nombres, apellidos, correo, usuario, contrasena, id_rol, puntos_acumulados, fecha_registro)
    | | VALUES ('$nombres', '$apellidos', '$correo', '$usuario', '$passwordHash', 4, 0, NOW())";

if (mysqli_query($db, $query)) {
    header('Location: login.php?res=exito');
    exit;
} else {
    $errores[] = "Error al crear la cuenta: " . mysqli_error($db);
}



ml>
```

carrito.php

```
if (isset($_GET['eliminar'])) {
    header('Location: carrito.php');
    exit;
}

// vaciar todo el carrito
if (isset($_GET['vaciar'])) {
    unset($_SESSION['carrito']);
    header('Location: carrito.php');
    exit;
}

// consultas y calculos para la vista

// Configuración de puntos
$resConf = mysqli_query($db, "SELECT valor_canje FROM configuracion WHERE id = 1");
$conf = mysqli_fetch_assoc($resConf);
$valor_monetario_punto = (float)($conf['valor_canje'] ?? 2.25);

// Datos del Usuario
$idUsuario = $_SESSION['id_usuario'] ?? null;
$puntosUsuario = 0.00;
if($idUsuario) {
    $resU = mysqli_query($db, "SELECT puntos_acumulados FROM usuarios WHERE id_usuario = $idUsuario");
    $uData = mysqli_fetch_assoc($resU);
    $puntosUsuario = (float)($uData['puntos_acumulados'] ?? 0.00);
}
```

admin > configuracion.php

```

$protocolo = isset($_SERVER['HTTPS']) && $_SERVER['HTTPS'] === 'on' ? 'https' : 'http';
$host = $_SERVER['HTTP_HOST'];
$url_base = $protocolo . "://" . $host . "/ferreteria_el_progreso/";

// cargar configuracion
$queryConfig = "SELECT * FROM configuracion WHERE id = 1";
$resConfig = mysqli_query($db, $queryConfig);
$config = mysqli_fetch_assoc($resConfig);
$puntos_por_minuto = (float)($config['puntos_minuto'] ?? 1.00);

// sincronizacion automatica
$updateSincro = "UPDATE usuarios
    | | | | | SET puntos_acumulados = puntos_acumulados + (TIMESTAMPDIFF(MINUTE, fecha_registro, NOW()) * $puntos_por_minut
    | | | | |   | fecha_registro = DATE_ADD(fecha_registro, INTERVAL TIMESTAMPDIFF(MINUTE, fecha_registro, NOW()) MINUTE)
    | | | | | WHERE TIMESTAMPDIFF(MINUTE, fecha_registro, NOW()) >= 1";
mysqli_query($db, $updateSincro);

// procesar formularios
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    if (isset($_POST['actualizar_reglas'])) {
        $p_m = mysqli_real_escape_string($db, $_POST['puntos_minuto']);
        $v_c = mysqli_real_escape_string($db, $_POST['valor_canje']);
        mysqli_query($db, "UPDATE configuracion SET puntos_minuto = '$p_m', valor_canje = '$v_c' WHERE id = 1");
        header("Location: configuracion.php?exito=1"); exit;
    }
```

admin > eliminar-usuario.php

```php
require __DIR__ . '/../includes/config/database.php';
$db = conectarDB();

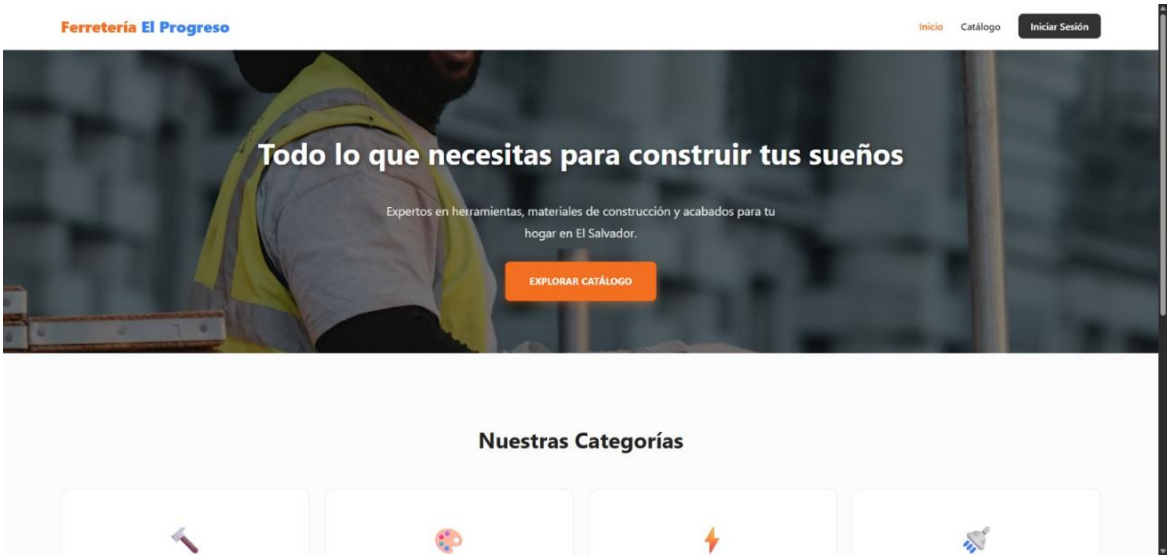
$id_a_borrar = filter_var($_GET['id'], FILTER_VALIDATE_INT);

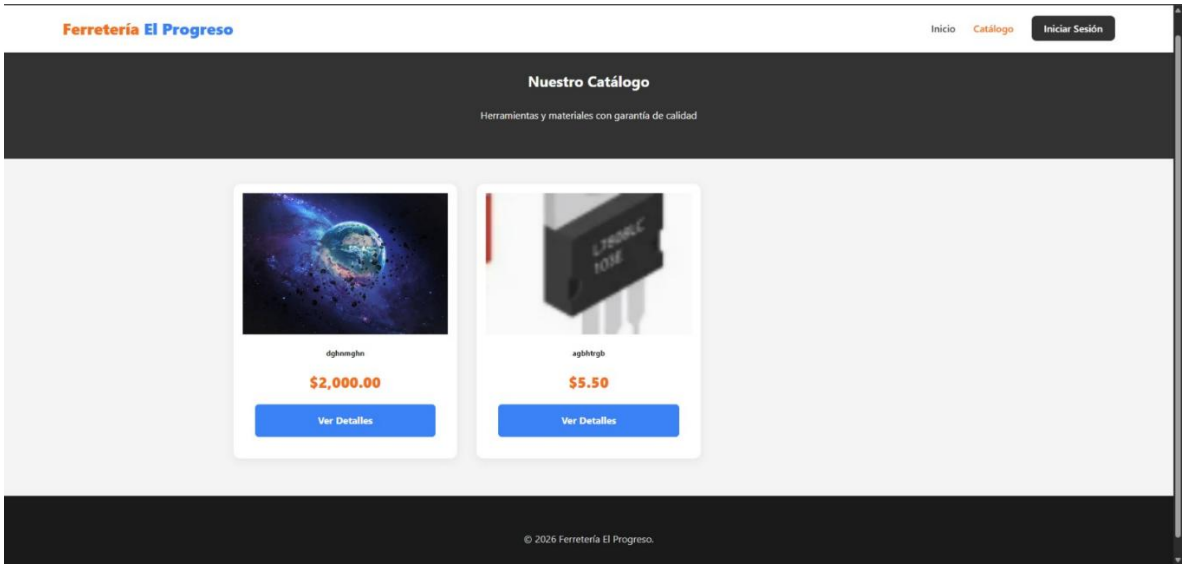
if($id_a_borrar) {
    // evita que el usuario se borre a si mismo
    if($id_a_borrar === (int)$_SESSION['id_usuario']) {
        header('Location: usuarios.php?resultado=3'); // Error: Autocantelación
        exit;
    }

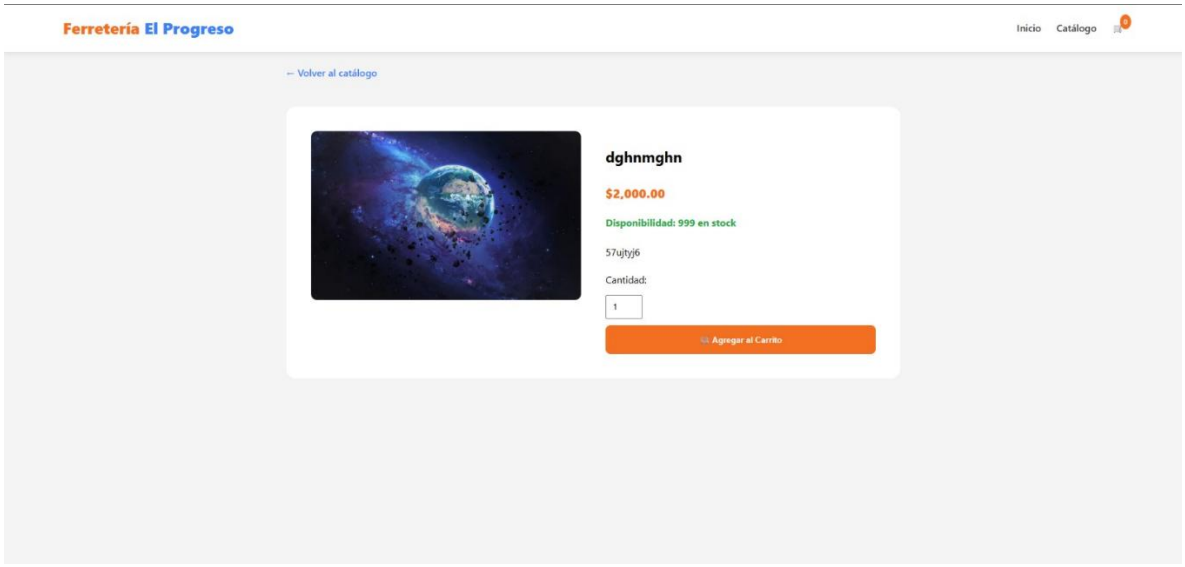
    // Intentar eliminación
    try {
        $query = "DELETE FROM usuarios WHERE id_usuario = {$id_a_borrar} LIMIT 1";
        $resultado = mysqli_query($db, $query);

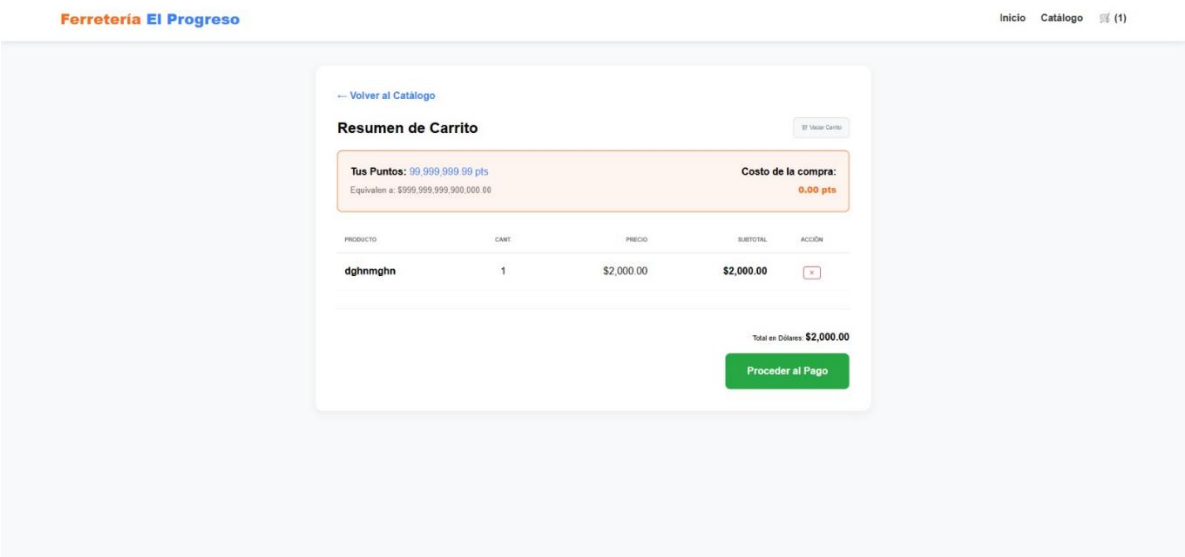
        if($resultado) {
            header('Location: usuarios.php?resultado=1'); // Éxito
        } else {
            header('Location: usuarios.php?resultado=3'); // Error general
        }
    } catch (mysqli_sql_exception $e) {
        // Si el usuario tiene ventas, la base de datos lanzará un error de "Foreign Key"
        header('Location: usuarios.php?resultado=3');
    }
} else {
```

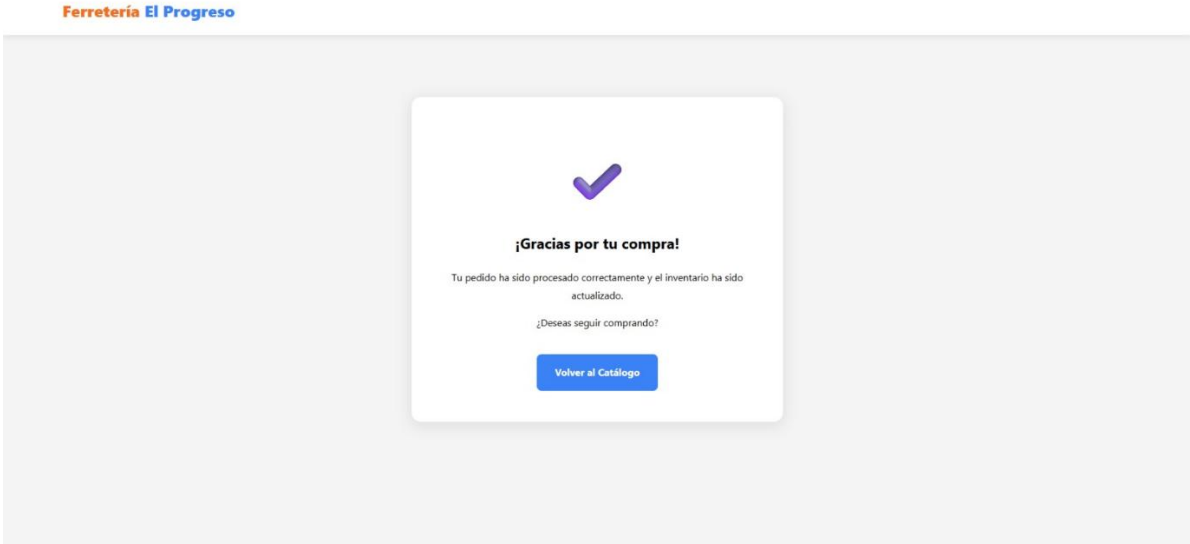
Pruebas del Sistema

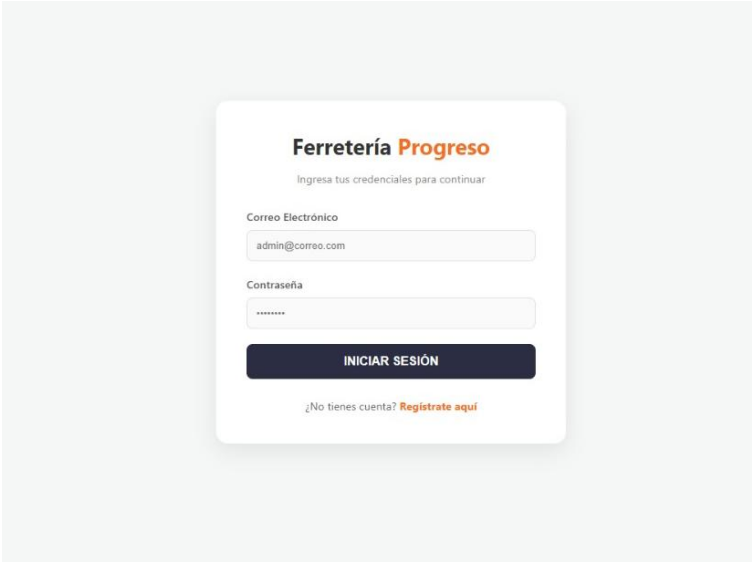


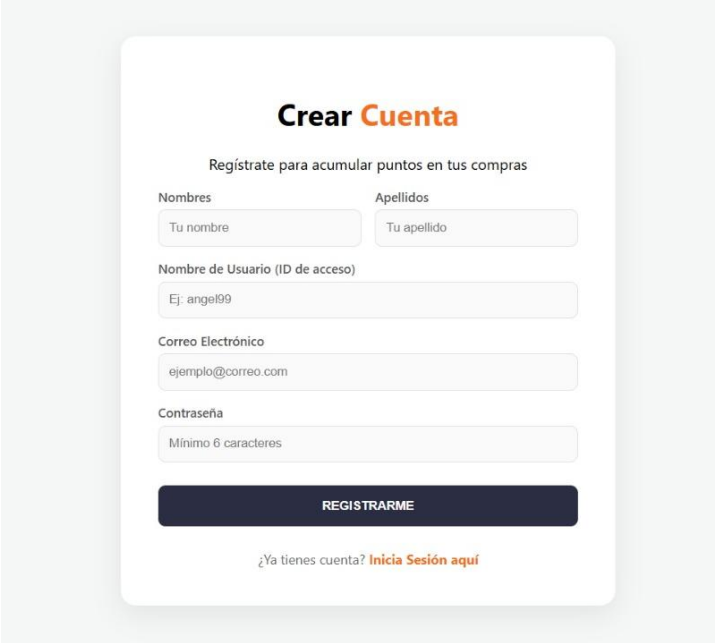


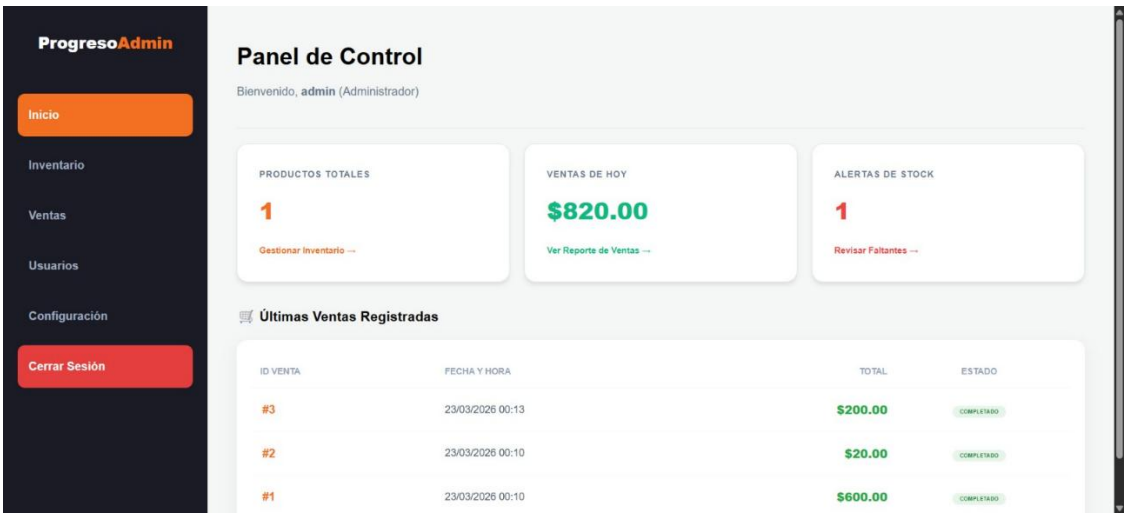

Ferretería El Progreso
Inicio
Catálogo
← Volver al catálogo
dghnmghn
$2,000.00
Disponibilidad: 999 en stock
57ujtyj6
Cantidad:
1
Agregar al Carrito

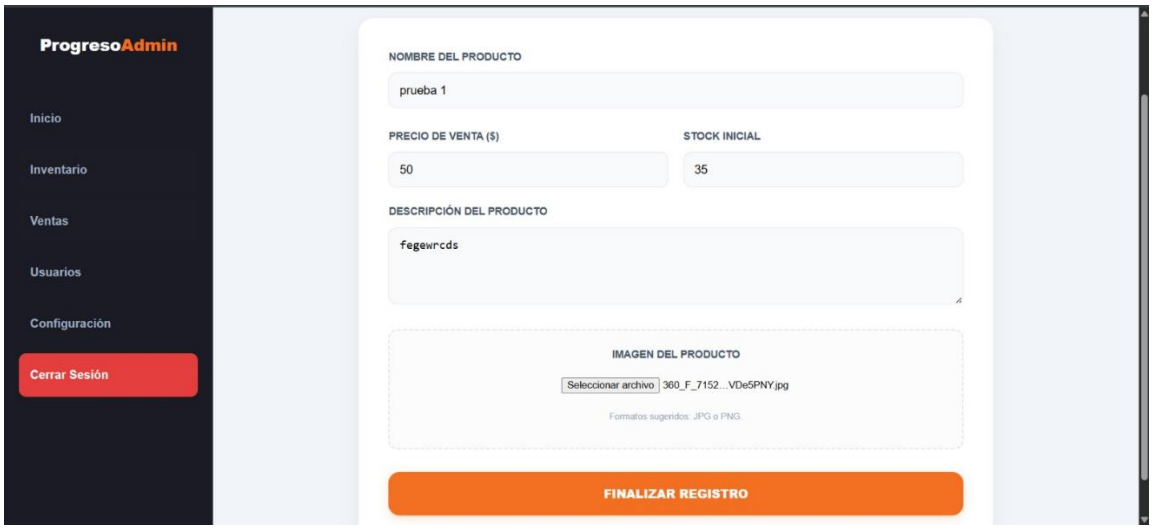

Ferretería El Progreso
Inicio
Catálogo
(1)
← Volver al Catálogo
Resumen de Carrito
Tus Puntos: 99,999,999.99 pts
Equivalen a: $999,999,999,900,000.00
Costo de la compra:
0.00 pts
PRODUCTO
CANT.
PRECIO
SUBTOTAL
ACCIÓN
dghnmghn
1
$2,000.00
$2,000.00
Total en Dólares: $2,000.00
Proceder al Pago

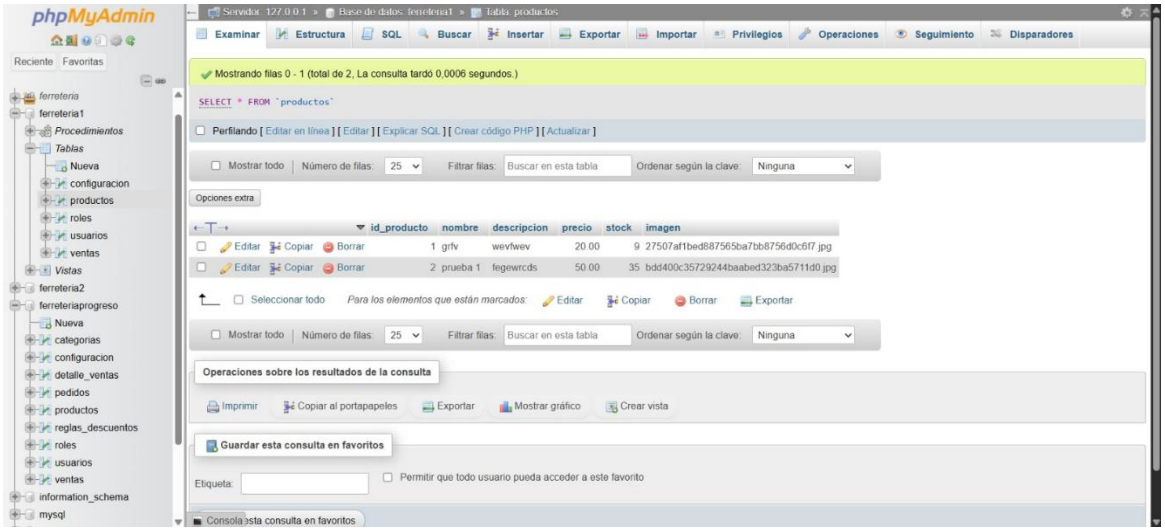

Ferretería El Progreso
¡Gracias por tu compra!
Tu pedido ha sido procesado correctamente y el inventario ha sido actualizado.
¿Deseas seguir comprando?
Volver al Catálogo

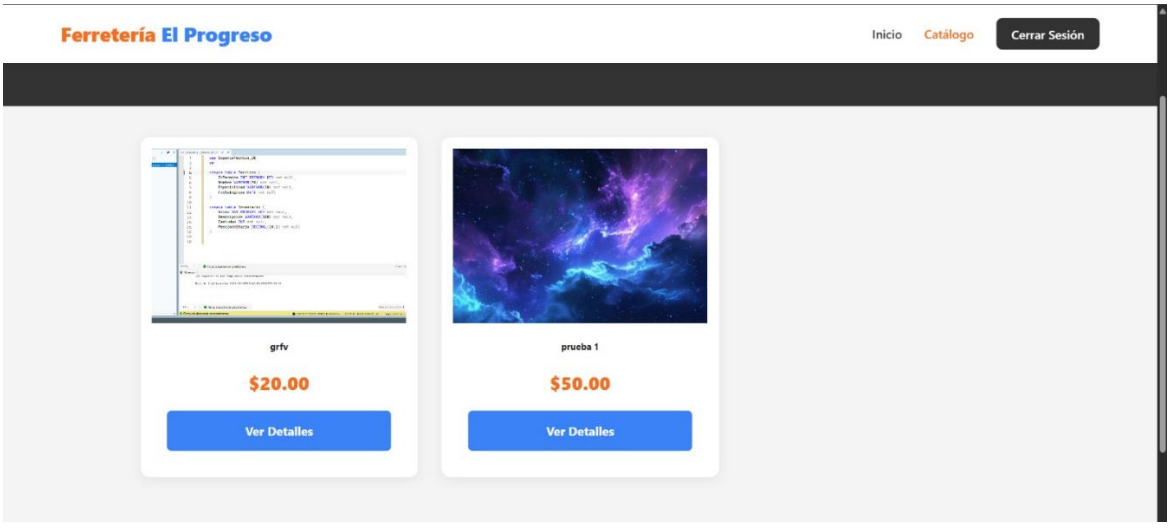

Ferretería Progreso
Ingresa tus credenciales para continuar
Correo Electrónico
admin@correo.com
Contraseña
INICIAR SESIÓN
¿No tienes cuenta? Regístrate aquí

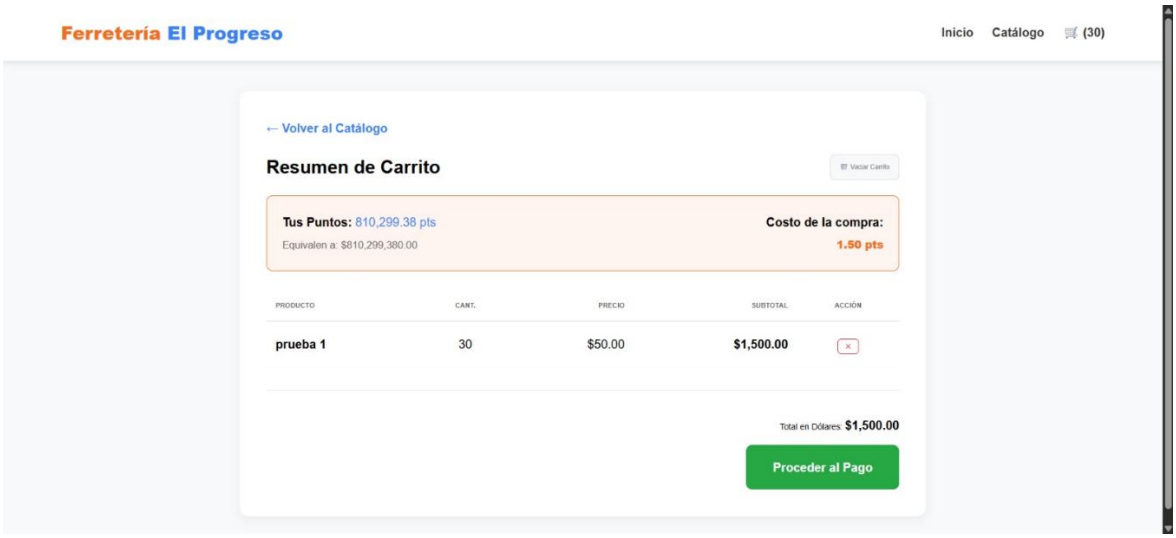

Crear Cuenta
Regístrate para acumular puntos en tus compras
Nombres
Tu nombre
Apellidos
Tu apellido
Nombre de Usuario (ID de acceso)
Ej: angel99
Correo Electrónico
ejemplo@correo.com
Contraseña
Mínimo 6 caracteres
REGISTRARME
¿Ya tienes cuenta? Inicia Sesión aquí

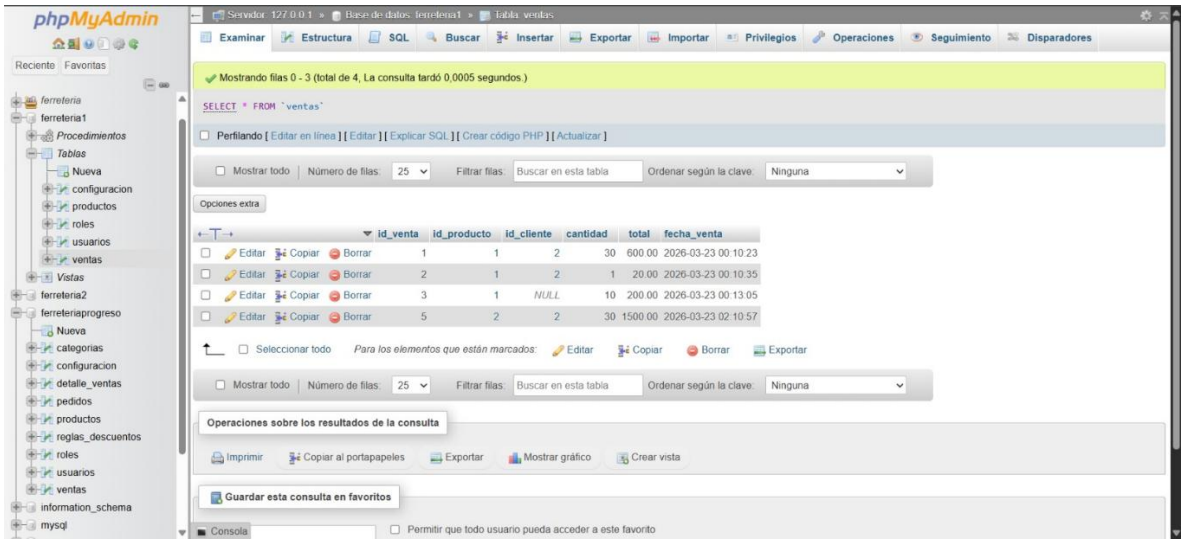

ProgresoAdmin
Inicio
Inventario
Ventas
Usuarios
Configuración
Cerrar Sesión
Panel de Control
Bienvenido, admin (Administrador)
PRODUCTOS TOTALES
1
Gestionar Inventario →
VENTAS DE HOY
$820.00
Ver Reporte de Ventas →
ALERTAS DE STOCK
1
Revisar Faltantes →
Últimas Ventas Registradas
ID VENTA
FECHA Y HORA
TOTAL
ESTADO
#3
23/03/2026 00:13
$200.00
COMPLETADO
#2
23/03/2026 00:10
$20.00
COMPLETADO
#1
23/03/2026 00:10
$600.00
COMPLETADO

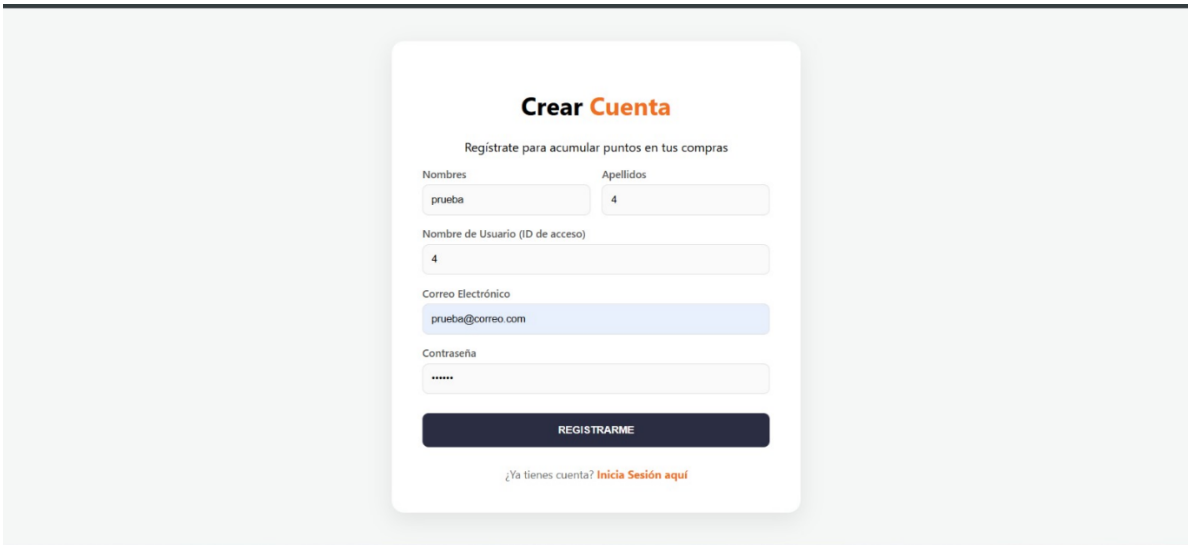

ProgresoAdmin
Inicio
Inventario
Ventas
Usuarios
Configuración
Cerrar Sesión
NOMBRE DEL PRODUCTO
prueba 1
PRECIO DE VENTA ($)
50
STOCK INICIAL
35
DESCRIPCIÓN DEL PRODUCTO
fegewrcds
IMAGEN DEL PRODUCTO
Seleccionar archivo
360_F_7152...VDe5PNY.jpg
Formatos sugeridos: JPG o PNG.
FINALIZAR REGISTRO

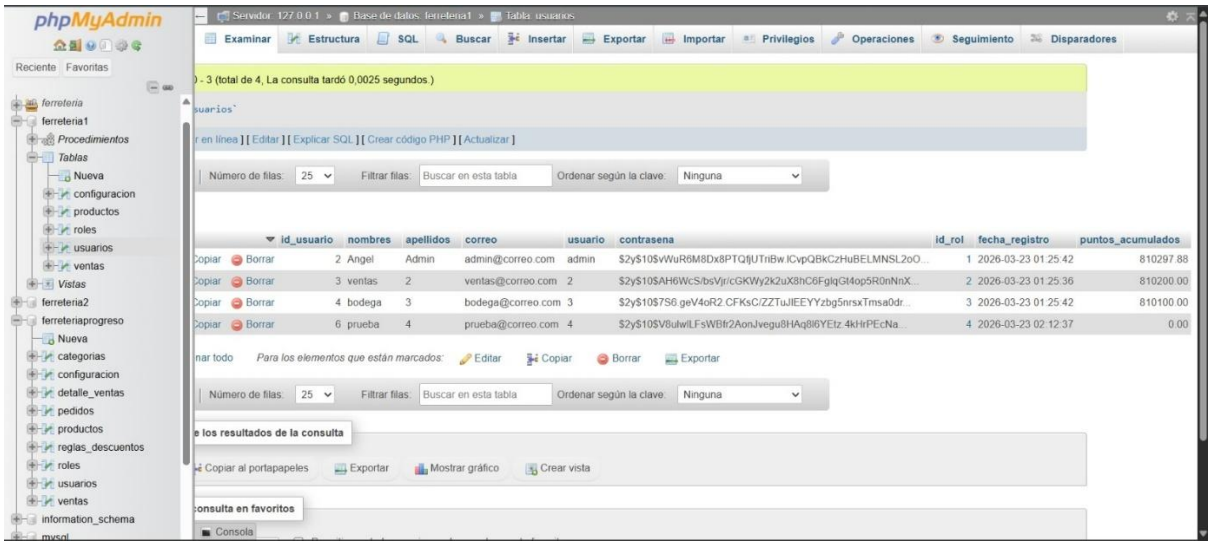

phpMyAdmin
Examinar
Estructura
SQL
Buscar
Insertar
Exportar
Importar
Privilegios
Operaciones
Seguimiento
Disparadores
Mostrando filas 0 - 1 (total de 2, La consulta tardó 0,0006 segundos.)
SELECT * FROM `productos`
id_producto
nombre
descripcion
precio
stock
imagen
1 grfv wevfwev 20.00 9 27507af1bed887565ba7bb8756d0c6f7.jpg
2 prueba 1 fegewrcds 50.00 35 bdd400c35729244baabed323ba5711d0.jpg
Operaciones sobre los resultados de la consulta
Guardar esta consulta en favoritos

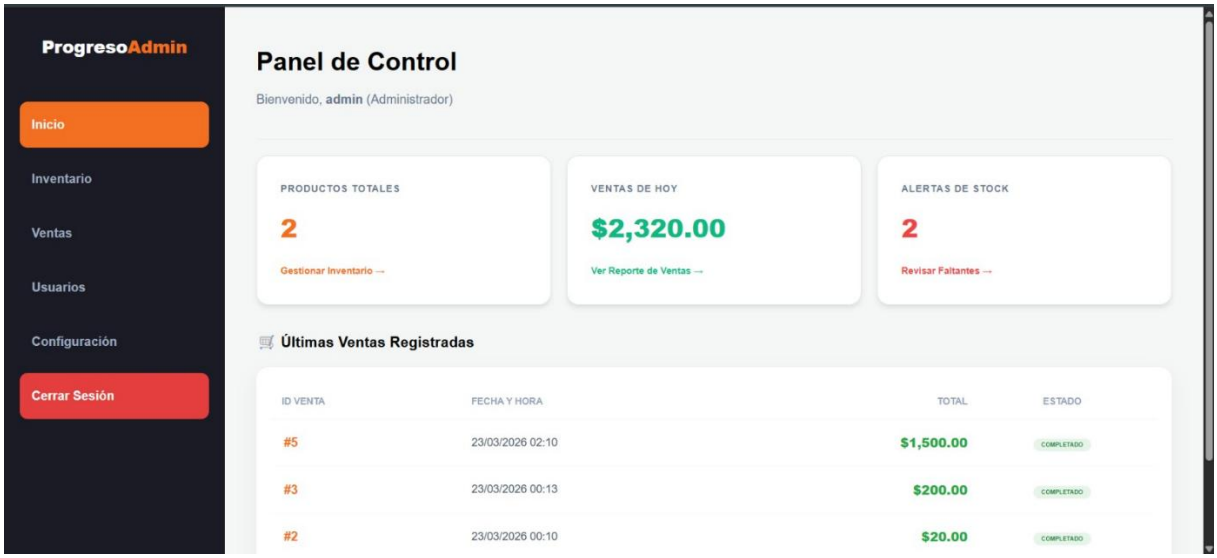

Ferretería El Progreso
Inicio
Catálogo
Cerrar Sesión
grfv
$20.00
Ver Detalles
prueba 1
$50.00
Ver Detalles

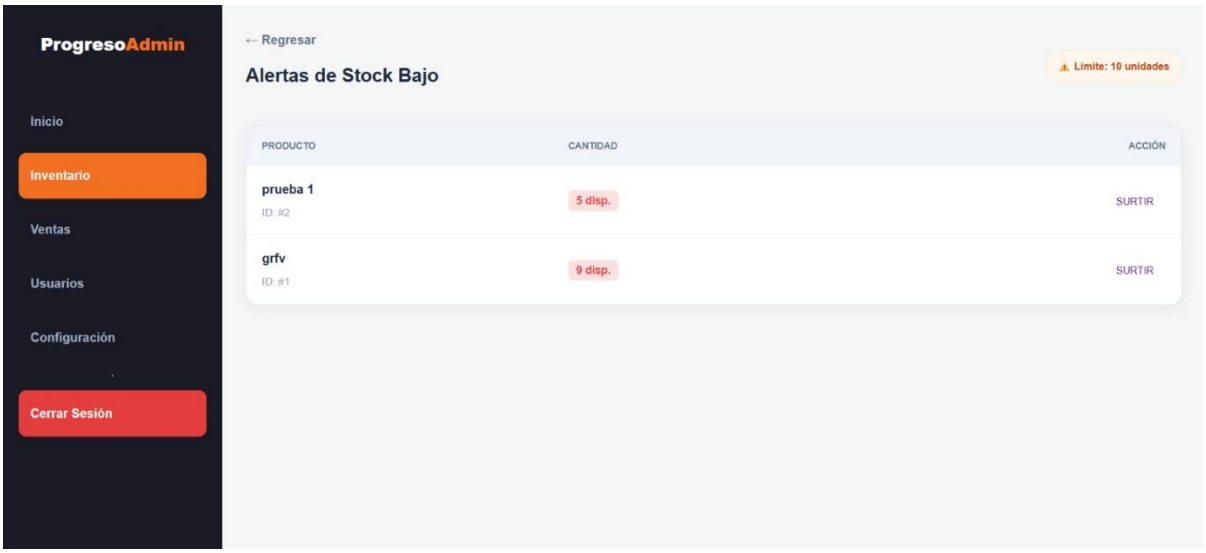

Ferretería El Progreso
Inicio
Catálogo
(30)
← Volver al Catálogo
Resumen de Carrito
Vaciar Carrito
Tus Puntos: 810,299.38 pts
Equivalen a: $810,299,380.00
Costo de la compra:
1.50 pts
PRODUCTO
CANT.
PRECIO
SUBTOTAL
ACCIÓN
prueba 1
30
$50.00
$1,500.00
Total en Dólares: $1,500.00
Proceder al Pago


phpMyAdmin
Reciente
Favoritas
ferreteria
ferreteria1
Procedimientos
Tablas
Nueva
configuracion
productos
roles
usuarios
ventas
Vistas
ferreteria2
ferreteriaprogreso
Nueva
categorias
configuracion
detalle_ventas
pedidos
productos
reglas_descuentos
roles
usuarios
ventas
information_schema
mysql
Servidor: 127.0.0.1 » Base de datos: ferreteria1 » Tabla: ventas
Examinar
Estructura
SQL
Buscar
Insertar
Exportar
Importar
Privilegios
Operaciones
Seguimiento
Disparadores
Mostrando filas 0 - 3 (total de 4, La consulta tardó 0,0005 segundos.)
SELECT * FROM `ventas`
Perfilando [Editar en línea] [Editar] [Explicar SQL] [Crear código PHP] [Actualizar]
Mostrar todo
Número de filas: 25
Filtrar filas: Buscar en esta tabla
Ordenar según la clave: Ninguna
Opciones extra
id_venta id_producto id_cliente cantidad total fecha_venta
Editar Copiar Borrar 1 1 2 30 600.00 2026-03-23 00:10:23
Editar Copiar Borrar 2 1 2 1 20.00 2026-03-23 00:10:35
Editar Copiar Borrar 3 1 NULL 10 200.00 2026-03-23 00:13:05
Editar Copiar Borrar 5 2 2 30 1500.00 2026-03-23 02:10:57
Seleccionar todo
Para los elementos que están marcados:
Editar
Copiar
Borrar
Exportar
Operaciones sobre los resultados de la consulta
Imprimir
Copiar al portapapeles
Exportar
Mostrar gráfico
Crear vista
Guardar esta consulta en favoritos
Consola
Permitir que todo usuario pueda acceder a este favorito


Crear Cuenta
Regístrate para acumular puntos en tus compras
Nombres
prueba
Apellidos
4
Nombre de Usuario (ID de acceso)
4
Correo Electrónico
prueba@correo.com
Contraseña
REGISTRARME
¿Ya tienes cuenta? Inicia Sesión aquí


phpMyAdmin
Reciente Favoritas
ferreteria
ferreteria1
Procedimientos
Tablas
Nueva
configuracion
productos
roles
usuarios
ventas
Vistas
ferreteria2
ferreteriaprogreso
Nueva
categorias
configuracion
detalle_ventas
pedidos
productos
reglas_descuentos
roles
usuarios
ventas
information_schema
mysql
Servidor: 127.0.0.1 » Base de datos: ferreteria1 » Tabla: usuarios
Examinar Estructura SQL Buscar Insertar Exportar Importar Privilegios Operaciones Seguimiento Disparadores
- 3 (total de 4, La consulta tardó 0,0025 segundos.)
[Editar] [Explicar SQL] [Crear código PHP] [Actualizar]
Número de filas: 25 Filtrar filas: Buscar en esta tabla Ordenar según la clave: Ninguna
id_usuario nombres apellidos correo usuario contrasena id_rol fecha_registro puntos_acumulados
Copiar Borrar 2 Angel Admin admin@correo.com admin $2y$10$vWuR6M8Dx8PTQfjUTnBw.lCvpQBkCzHuBELMNSL2oO... 1 2026-03-23 01:25:42 810297.88
Copiar Borrar 3 ventas 2 ventas@correo.com 2 $2y$10$AH6WcS/bsVjr/cGKWy2k2uX8hC6FglqGt4op5R0nNnX... 2 2026-03-23 01:25:36 810200.00
Copiar Borrar 4 bodega 3 bodega@correo.com 3 $2y$10$7S6.geV4oR2.CFKsC/ZZTuJIEEYYzbg5nrsxTmsa0dr... 3 2026-03-23 01:25:42 810100.00
Copiar Borrar 6 prueba 4 prueba@correo.com 4 $2y$10$V8ulwlLFsWBfr2AonJvegu8HAq8l6YEtz.4kHrPEcNa... 4 2026-03-23 02:12:37 0.00
Para los elementos que están marcados: Editar Copiar Borrar Exportar
Número de filas: 25 Filtrar filas: Buscar en esta tabla Ordenar según la clave: Ninguna
Copiar al portapapeles Exportar Mostrar gráfico Crear vista
Consola


ProgresoAdmin
Inicio
Inventario
Ventas
Usuarios
Configuración
Cerrar Sesión
Panel de Control
Bienvenido, admin (Administrador)
PRODUCTOS TOTALES
2
Gestionar Inventario →
VENTAS DE HOY
$2,320.00
Ver Reporte de Ventas →
ALERTAS DE STOCK
2
Revisar Faltantes →
Últimas Ventas Registradas
ID VENTA FECHA Y HORA TOTAL ESTADO
#5 23/03/2026 02:10 $1,500.00 COMPLETADO
#3 23/03/2026 00:13 $200.00 COMPLETADO
#2 23/03/2026 00:10 $20.00 COMPLETADO


ProgresoAdmin
Inicio
Inventario
Ventas
Usuarios
Configuración
Cerrar Sesión
← Regresar
Alertas de Stock Bajo
Límite: 10 unidades
PRODUCTO CANTIDAD ACCIÓN
prueba 1
ID: #2
5 disp.
SURTIR
grfv
ID: #1
9 disp.
SURTIR

Conclusiones

El desarrollo integral de este sistema permitió la consolidación de conocimientos teóricos y prácticos en la creación de soluciones para la gestión de inventario y capital. Mediante el uso de herramientas de estándar industrial como XAMPP y Visual Studio Code, se logró diseñar una arquitectura robusta basada en roles de usuario, garantizando que cada operario interactúe únicamente con los módulos correspondientes a su nivel de acceso. Un aprendizaje crítico durante este proceso fue la implementación de lógica avanzada en el motor de base de datos a través de Vistas y Procedimientos Almacenados, elementos que no solo optimizan el rendimiento de las consultas, sino que elevan la profesionalidad del software al delegar procesos complejos directamente al servidor.

Uno de los hitos técnicos más relevantes fue el fortalecimiento de la integridad referencial y la persistencia de los datos. Se aplicaron restricciones de tipo ON DELETE SET NULL, una estrategia fundamental que permite que, ante la eliminación de un registro "padre" (el usuario), los registros "hijos" (las ventas) permanezcan huérfanos pero intactos. Esta configuración es vital para la contabilidad empresarial, ya que evita la pérdida de información histórica en cadena y mantiene la coherencia de los reportes financieros sin importar los cambios en la nómina de personal.

No obstante, el proyecto no estuvo exento de dificultades técnicas que pusieron a prueba la capacidad de resolución de problemas del equipo. Desde inconsistencias iniciales en la configuración del servidor local MariaDB de XAMPP, hasta desafíos críticos como la pérdida de código fuente por errores humanos durante la fase de desarrollo. Estas experiencias subrayaron la importancia de implementar protocolos de respaldo y control de versiones. Finalmente, se concluye que una gestión de bases de datos eficiente es el pilar de cualquier organización; no se trata solo de almacenar información, sino de garantizar que los registros históricos y actuales conserven su integridad y disponibilidad, asegurando así el correcto funcionamiento y la toma de decisiones estratégicas dentro de cualquier entidad económica.

Recomendaciones

Para garantizar la continuidad y el crecimiento del sistema "Ferretería El Progreso", se recomienda encarecidamente establecer un protocolo de copias de seguridad automatizadas con periodicidad diaria o mensual, asegurando la conservación integral de los datos históricos ante cualquier percance técnico o fallo del servidor. Asimismo, es fundamental implementar el uso de sistemas de control de versiones como Git desde las etapas iniciales del desarrollo; esta práctica, que se identificó como una necesidad tras la pérdida accidental de código por error humano, permitiría rastrear cambios y recuperar versiones estables del software de manera inmediata. Debido a las limitaciones de permisos encontradas en la infraestructura de hosting gratuito, se sugiere la migración hacia un entorno de hosting de pago o VPS que permita reintegrar las Vistas y Procedimientos Almacenados directamente en el motor de base de datos, optimizando así el rendimiento global del sistema.

De igual manera, se debe priorizar el robustecimiento de la seguridad mediante la migración a sentencias preparadas (Prepared Statements) en PHP para blindar la base de datos contra ataques de inyección SQL, una medida de protección estándar que debe acompañar a todo sistema de gestión profesional. Complementariamente, es vital proporcionar un mantenimiento técnico constante y actualizaciones periódicas al código y a las dependencias del servidor para prevenir la obsolescencia y corregir vulnerabilidades. Finalmente, se recomienda una capacitación técnica constante al personal encargado de operar la plataforma, asegurando que los usuarios finales estén plenamente actualizados sobre las funciones críticas y el flujo de trabajo del sistema, lo cual reduce errores operativos y garantiza que la herramienta cumpla eficazmente con su propósito de gestión empresarial.

Bibliografía

— Ana, Q. A. (s.f.). *Mentores Tech*.

13 key objectives of inventory management. (s.f.). Obtenido de https://www.fishbowlinventory.com/blog/10-goals-of-good-inventory-management

AltexSoft. (16 de 05 de 2023). Obtenido de https://www.altexsoft.com/blog/comparing-database-management-systems-mysql-postgresql-mssql-server-mongodb-elasticsearch-and-others/

Blog para fabricantes y distribuidores. (08 de 08 de 2025). Obtenido de https://www.mrpeasy.com/blog/es/costos-de-inventario/

Gomstyn, A., & Jonker, A. (27 de 11 de 2025). *Ibm.com*. Recuperado el 3 de 3 de 2026, de Ibm.com: https://www.ibm.com/mx-es/think/topics/database-normalization

Hpe.com. (s.f.). Obtenido de https://www.hpe.com/lamerica/es/what-is/it-security.html

Ibm.com. (7 de 10 de 2025). Obtenido de https://www.ibm.com/docs/es/i/7.5.0?topic=concepts-tables

InetSoft. (01 de 01 de 2012). Obtenido de https://www.inetsoft.com/info/database-fields-and-types/

Navarro, S. (09 de 08 de 2022). *KeepCoding Bootcamps*. Obtenido de https://keepcoding.io/blog/llaves-primarias-y-foraneas/

Prieto, E. (19 de 09 de 2022). *SNHU*. (S. N. University, Editor) Obtenido de https://es.snhu.edu/blog/por-que-es-importante-el-control-de-inventarios

Turovski, M. (25 de 03 de 2024). *Blog para fabricantes y distribuidores*. Obtenido de https://www.mrpeasy.com/blog/es/tipos-de-inventario/